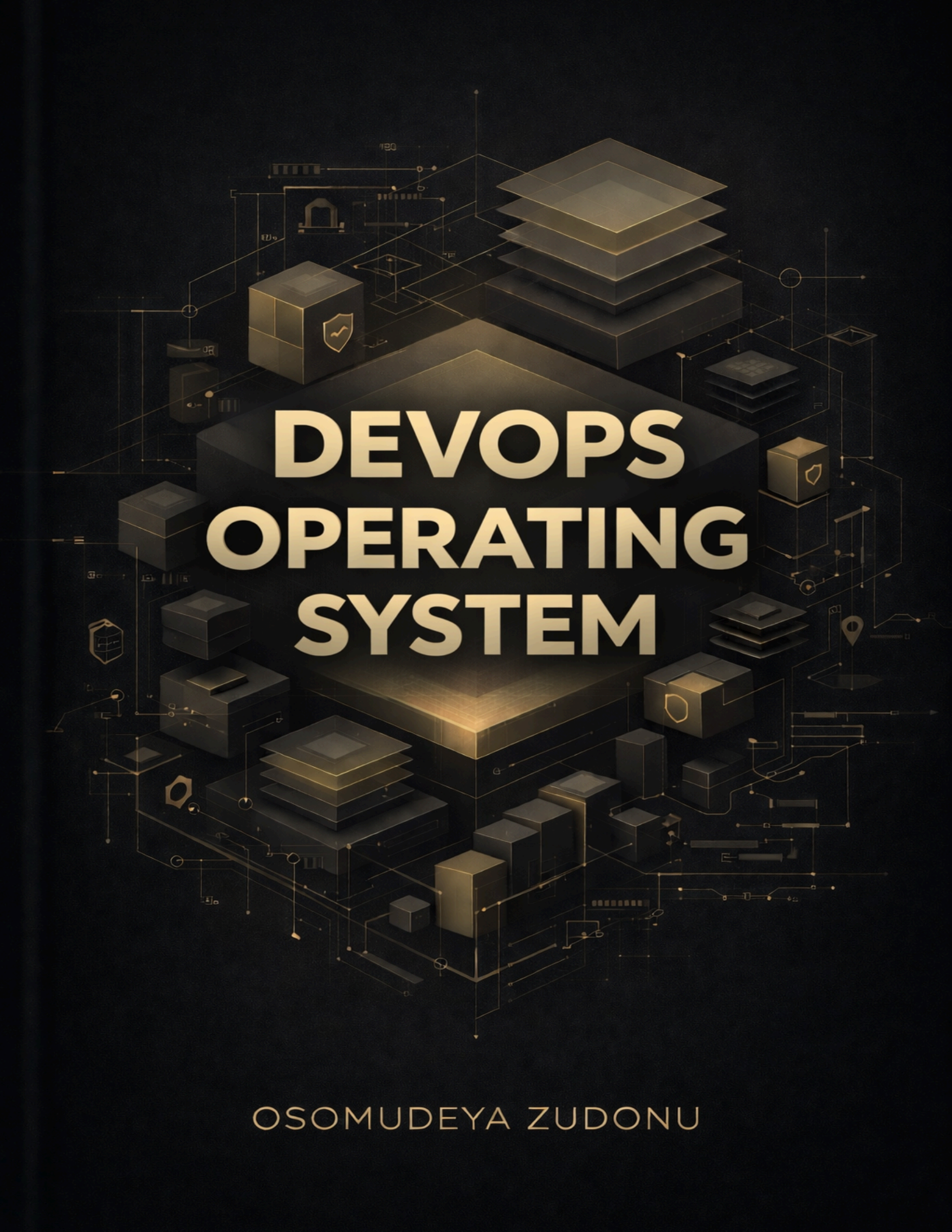




DEVOPS OPERATING SYSTEM

OSOMUDEYA ZUDONU

Before You Begin

Code Disclaimer

This ebook includes code samples, Terraform configurations, and deployment scripts tested at the time of writing. However:

- **Version changes** can introduce compatibility issues
- **AWS updates** may require configuration tweaks
- **Environment constraints** (permissions, quotas) may differ
- **Regional availability** can impact certain services

If something fails:

1. Read the error message, it often identifies the issue
2. Confirm version alignment or upgrade to a stable release
3. Review AWS documentation for recent changes
4. Validate account limits and permissions
5. Modify the code to fit your setup

Focus on understanding: The objective is learning, not copy-pasting. Knowing *why* it works is more valuable than getting it right on the first attempt.

SECTION 9

GitOps with ArgoCD

The GitOps Gap

You've spent weeks building H-Game on production AWS infrastructure. You've learned Kubernetes, Terraform, EKS, security scanning, CI/CD, and observability. Your application is deployed, secure, automated, and monitored.

But there's a critical gap.

You're still deploying manually with `kubectl apply`. Your CI/CD pipeline builds images and pushes to ECR, but deployment is manual. You have no single source of truth for infrastructure state. You can't easily rollback. You can't track who deployed what, when.

You've learned how to build, deploy, secure, and observe. Now it's time to automate deployment management.

This section adds GitOps to your infrastructure. You'll implement:

- **ArgoCD installation** - GitOps controller for Kubernetes
- **Git-based deployments** - Git as a single source of truth
- **Multi-environment management** - Dev, staging, production from one repo
- **Advanced patterns** - Sync waves, hooks, progressive delivery
- **CI/CD integration** - Complete automation from code to cluster

All open source. All integrated. All production ready.

Prerequisites

Before starting Section 9, ensure you have:

Required Skills:

- Completed Section 8 (O1-O5) - Observability & Monitoring
- Completed Section 7 (P1-P4) - CI/CD with GitHub Actions
- Completed Section 6 (D1-D5) - DevSecOps and security scanning
- Completed Section 5 (T1-T10) - Terraform & AWS infrastructure
- Completed Section 4 (K1-K9) - Kubernetes fundamentals
- AWS account with EKS cluster running
- GitHub account with repository access

Required Tools:

- AWS CLI installed and configured
- kubectl installed and configured
- Helm installed (for ArgoCD installation)
- Git installed
- ArgoCD CLI installed (instructions in G1)

Recommended Tools:

- IDE/Code Editor (VS Code, Cursor, or similar)
- GitHub repository for GitOps manifests

Note: GitOps concepts are introduced in this section. Previous sections provide the foundation (K8s, AWS, CI/CD, application code).

Section 9 Tasks

Task	Focus	Time	Cost
G1	ArgoCD Installation & GitOps Basics	3-4h	Free
G2	Application Deployment with ArgoCD	3-4h	Free
G3	Multi-Environment Strategy	3-4h	Free
G4	Advanced Patterns (Sync Waves, Hooks)	2-3h	Free
G5	Production Workflow (CI/CD Integration)	2-3h	Free
Total		13-18 h	\$0

Cost Transparency

Reality check before you start:

All tools in Section 9 are open source and free:

- **ArgoCD:** Free (runs in your cluster)
- **Kustomize:** Free (built into kubectl)
- **GitHub:** Free (public repos) or included in existing plan

Storage costs:

- Git repository: Minimal (manifests are small text files)
- ArgoCD metadata: ~100-200MB storage (included in EKS)

Total cost: \$0

All GitOps tools run in your existing Kubernetes cluster. No additional AWS services required.

The One Thing That Makes This Click

GitOps = version control for infrastructure. Just like developers use Git to track code changes, GitOps uses Git to track infrastructure changes. Every deployment, every config change, every rollback is a Git commit.

Without GitOps:

- Manual `kubectl apply` commands
- No audit trail of who deployed what
- Hard to rollback (what was the previous state?)
- Cluster drift (manual changes not in Git)
- No single source of truth

With GitOps:

- Git commit triggers deployment
- Full audit trail in Git history
- Easy rollback (Git revert)
- Self-healing (detects and corrects drift)
- Git is the single source of truth

What You'll Build

Week 1: Foundation (G1-G2)

- ArgoCD installation on EKS
- GitOps repository structure
- Deploy H-Game application via ArgoCD
- Understand GitOps principles

Week 2: Multi-Environment (G3)

- Dev, staging, production environments
- Customize overlays for environment-specific configs
- Environment isolation with namespaces

Week 3: Advanced Patterns (G4-G5)

- Sync waves for ordered deployment
- Pre-sync and post-sync hooks
- Rollback strategies
- CI/CD + GitOps integration

Complete GitOps Workflow:

```
Developer → Git commit → GitHub Actions (build/test)
      ↓
      Update GitOps repo (image tags)
      ↓
      ArgoCD detects change
      ↓
      Deploys to Kubernetes
      ↓
      Prometheus monitors (Section 8)
```

GitOps Philosophy

Core Principles:

1. Git as Single Source of Truth
 - All Kubernetes manifests in Git
 - All infrastructure configs in Git
 - Git history = deployment history
2. Declarative Everything
 - Describe desired state, not steps
 - ArgoCD applies changes automatically
 - No imperative commands (`kubectl apply`)
3. Automated Synchronization
 - ArgoCD continuously monitors Git
 - Detects changes automatically
 - Syncs cluster to match Git state
4. Self-Healing Systems
 - Detects cluster drift (manual changes)
 - Automatically corrects to match Git
 - Cluster always matches Git state
5. Audit Trail Built-In
 - Every deployment = Git commit
 - Who deployed what, when (Git author)
 - Easy to see what changed and why

What's Different from Section 7?

Section 7 (CI/CD):

- Builds Docker images
- Pushes to ECR
- Runs tests and security scans
- Updates Kubernetes directly (kubectl)

Section 9 (GitOps):

- CI/CD builds images and updates Git
- ArgoCD monitors Git and deploys
- No kubectl from CI/CD
- Git is the deployment interface

Key Difference:

- Section 7: CI/CD has direct cluster access
- Section 9: CI/CD only touches Git, ArgoCD touches cluster

Why This Matters:

- Better security (CI/CD doesn't need cluster access)
- Better audit trail (Git history)
- Better rollback (Git revert)
- Better collaboration (Git workflows)

Learning Outcomes

By the end of Section 9, you will:

1. Understand GitOps principles
 - Git as single source of truth
 - Declarative infrastructure
 - Automated synchronization
 - Self-healing systems
2. Install and configure ArgoCD
 - Deploy ArgoCD on EKS
 - Access ArgoCD UI and CLI
 - Configure Git repositories

- Create ArgoCD Applications
- 3. Deploy applications with ArgoCD
 - Structure GitOps repository
 - Create Application manifests
 - Configure sync policies
 - Monitor deployment status
- 4. Manage multi-environment deployments
 - Customize base and overlays
 - Environment-specific configurations
 - Namespace isolation
 - Environment promotion workflows
- 5. Implement advanced GitOps patterns
 - Sync waves for ordered deployment
 - Pre-sync and post-sync hooks
 - Rollback strategies
 - Progressive delivery
- 6. Integrate CI/CD with GitOps
 - Update GitOps repo from CI/CD
 - ArgoCD auto-sync on Git changes
 - Complete automation pipeline
 - Deployment annotations

Success Metrics

You'll know Section 9 is complete when:

- ArgoCD is installed and accessible
- H-Game application deployed via ArgoCD
- Multi-environment setup working (dev/staging/prod)
- Git commit triggers automatic deployment
- Rollback works via Git revert
- CI/CD pipeline updates GitOps repo
- Full audit trail in Git history

Career Impact

GitOps is essential for modern DevOps roles:

Why employers care:

- Production ready deployment patterns
- Security best practices (no kubectl from CI)
- Audit compliance (Git history)
- Multi environment management
- Team collaboration (Git workflows)

Interview talking points:

- "I implemented GitOps with ArgoCD for multi-environment deployments"
- "I separated CI/CD from deployment (CI updates Git, ArgoCD deploys)"
- "I used Kustomize overlays for environment specific configs"
- "I implemented sync waves and hooks for ordered deployments"
- "I integrated GitHub Actions with ArgoCD for complete automation"

Real-world applications:

- Any Kubernetes deployment
- Multi-environment management
- Compliance and audit requirements
- Team collaboration workflows
- Production-grade deployments

Common Questions

Q: Do I need a separate Git repository for GitOps?

A: Yes, create a dedicated repository for Kubernetes manifests. This keeps application code separate from deployment configs.

Q: Can I use Helm charts with ArgoCD?

A: Yes, ArgoCD supports Helm charts, Kustomize, and raw YAML. This section focuses on Kustomize for simplicity.

Q: What if I already have deployments running?

A: ArgoCD can adopt existing resources. You'll move manifests to Git and create Applications pointing to them.

Q: How does this work with Section 7 CI/CD?

A: CI/CD builds images and updates image tags in GitOps repo. ArgoCD detects changes and deploys. CI/CD never touches Kubernetes directly.

Q: What about secrets?

A: Don't commit secrets to Git. Use External Secrets Operator (from Section 6) or Sealed Secrets. ArgoCD will sync secret manifests, but values come from secure sources.

Q: Can I use this with multiple clusters?

A: Yes, ArgoCD supports multi-cluster deployments. One ArgoCD instance can manage multiple Kubernetes clusters.

Next Steps

Ready to start? Begin with G1: ArgoCD Installation & GitOps Basics

You'll install ArgoCD, understand GitOps principles, and create your first GitOps repository structure.

Section 9 Summary

What you'll build:

- ArgoCD installation on EKS
- GitOps repository with H-Game manifests
- Multi-environment deployments (dev/staging/prod)
- Advanced patterns (sync waves, hooks, rollback)
- CI/CD + GitOps integration

Complete workflow:

```
Developer → Git commit → GitHub Actions (build/test)
                        ↓
                    Update GitOps repo
                        ↓
                ArgoCD detects change
                        ↓
                Deploy to Kubernetes
                        ↓
            Prometheus monitors (Section 8)
```

Cost: \$0 (all open-source)

Production benefits:

- Git as single source of truth
- Full audit trail (who deployed what, when)
- Easy rollback (Git revert)
- Self-healing (drift correction)
- Multi-environment management
- Secure (no kubectl access needed for CI)

G1: ArgoCD Installation & GitOps Basics

Time: 3-4 hours | **Cost:** Free

Mission Brief

Install ArgoCD on your EKS cluster and understand GitOps principles. Learn why Git becomes the single source of truth for all deployments, replacing manual `kubectl apply` commands with automated Git-based deployments.

Outcomes:

- Understand GitOps principles and benefits
- Install ArgoCD on EKS cluster
- Access ArgoCD UI and CLI
- Create GitOps repository structure
- Deploy first application via ArgoCD
- Understand sync policies and drift detection

Prerequisites: Section 8 (O1-O5) complete, EKS cluster running, H-Game deployed, GitHub account

Important Note

Code may need adjustments: Examples are based on specific versions and environments. If errors occur:

1. Confirm version compatibility (Terraform, Kubernetes, AWS CLI)
2. Verify account permissions and quotas
3. Read error messages they usually indicate the fix
4. Check official documentation for your version
5. Use the troubleshooting section if available

Expect differences: Cloud services and tool versions evolve. Treat these examples as guidance and adapt them to your environment.

The One Thing That Makes This Click

GitOps = version control for infrastructure. Just like developers use Git to track code changes, GitOps uses Git to track infrastructure changes. Every deployment, every config change, every rollback is a Git commit.

Without GitOps:

- Manual `kubectl apply` commands
- No audit trail of who deployed what
- Hard to rollback (what was the previous state?)
- Cluster drift (manual changes not in Git)
- No single source of truth

With GitOps:

- Git commit triggers deployment
- Full audit trail in Git history
- Easy rollback (Git revert)
- Self-healing (detects and corrects drift)
- Git is the single source of truth

In essence: Git becomes your deployment interface. Everything else (ArgoCD, Applications, sync policies) is just details.

The Mental Model Shift:

BEFORE G1:

- "I deploy with `kubectl apply`"
- "I hope I remember what I changed"
- "Rollback? What was the previous state?"

AFTER G1:

- "I deploy with `git commit`"
- "Git history shows every change"
- "Rollback? Just `git revert`"

This is the difference between "manual deployment" and "declarative deployment."

Now let's see how GitOps works...

COMPLETE PREREQUISITES SECTION

```
## Prerequisites & Pre-Flight Checklist
```

```
Time: 10-15 minutes
```

```
CRITICAL: Complete this BEFORE starting ArgoCD installation. Missing prerequisites will cause failures.
```

```
### Step 0.1: Verify Infrastructure Exists
```

```
If you destroyed AWS resources (Section 8 cleanup), you MUST recreate them first:
```

```
# Check if EKS cluster exists
```

```
aws eks list-clusters --region us-east-1
```

```
# Expected output:
```

```
# {
```

```
#   "clusters": [
```

```
#     "h-game-prod" # or your cluster name
```

```
#   ]
```

```
# }
```

If cluster doesn't exist:

```
# Option 1: Recreate via Terraform (recommended)
```

```
cd h-game-terraform
```

```
terraform apply
```

```
# Option 2: Follow detailed guide
```

```
# See: h-game-terraform/RECREATION_GUIDE.md
```

Step 0.2: Verify kubectl Configuration

```
# Test kubectl connection
kubectl get nodes

# Expected output:
# NAME                                STATUS    ROLES    AGE    VERSION
# ip-10-0-1-123.ec2.internal          Ready    <none>    5m     v1.28.x
# ip-10-0-2-456.ec2.internal          Ready    <none>    5m     v1.28.x

# If this fails, configure kubectl:
CLUSTER_NAME=$(cd h-game-terraform && terraform output -raw
eks_cluster_name)
aws eks update-kubeconfig --name $CLUSTER_NAME --region us-east-1
```

Step 0.3: Get ECR Registry URL

```
# Get AWS account ID
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output
text)
echo "Your Account ID: $ACCOUNT_ID"

# ECR registry format
ECR_REGISTRY="${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com"
echo "ECR Registry: $ECR_REGISTRY"

# Save for later use
export ECR_REGISTRY=$ECR_REGISTRY
echo "export ECR_REGISTRY=$ECR_REGISTRY" >> ~/.zshrc # or ~/.bashrc
```

Expected output:

```
Your Account ID: 334091769766
ECR Registry: 334091769766.dkr.ecr.us-east-1.amazonaws.com
```

Step 0.4: Verify ECR Repositories Exist

```
# Check backend repository
aws ecr describe-repositories --repository-names h-game-backend
--region us-east-1

# Check frontend repository
aws ecr describe-repositories --repository-names h-game-frontend
--region us-east-1

# Expected output (for each):
# {
#   "repositories": [
#     {
#       "repositoryName": "h-game-backend",
#       "repositoryUri":
# "334091769766.dkr.ecr.us-east-1.amazonaws.com/h-game-backend",
#       ...
#     }
#   ]
# }
```

If repositories don't exist:

```
# Create backend repository
aws ecr create-repository \
  --repository-name h-game-backend \
  --region us-east-1 \
  --image-scanning-configuration scanOnPush=true

# Create frontend repository
aws ecr create-repository \
  --repository-name h-game-frontend \
  --region us-east-1 \
  --image-scanning-configuration scanOnPush=true
```

Or recreate via Terraform:

```
cd h-game-terraform
terraform apply -target=aws_ecr_repository.backend
               -target=aws_ecr_repository.frontend
```

Step 0.5: Create Kubernetes Namespace

```
# Create h-game namespace (if it doesn't exist)
kubectl create namespace h-game --dry-run=client -o yaml | kubectl
apply -f -

# Verify namespace exists
kubectl get namespace h-game

# Expected output:
# NAME      STATUS   AGE
# h-game    Active   5s
```

Step 0.6: Create Required Secrets

Get database credentials from Terraform:

```
# Navigate to Terraform directory
cd h-game-terraform

# Get RDS endpoint (format: hostname:port)
RDS_ENDPOINT=$(terraform output -raw rds_endpoint)
echo "RDS Endpoint: $RDS_ENDPOINT"

# Extract hostname (remove :port)
RDS_HOST=$(echo $RDS_ENDPOINT | cut -d: -f1)
RDS_PORT=$(echo $RDS_ENDPOINT | cut -d: -f2)
echo "RDS Host: $RDS_HOST"
echo "RDS Port: $RDS_PORT"
```

```

# Get database credentials from Terraform variables
# Note: These come from terraform.tfvars (not outputs)
# If you don't have terraform.tfvars, check terraform.tfvars.example

# Option 1: Read from terraform.tfvars (if exists)
if [ -f terraform.tfvars ]; then
    RDS_USER=$(grep rds_username terraform.tfvars | cut -d'"' -f2)
    RDS_PASSWORD=$(grep rds_password terraform.tfvars | cut -d'"' -f2)
    RDS_DB=$(grep rds_db_name terraform.tfvars | cut -d'"' -f2)
else
    echo " terraform.tfvars not found. Using defaults from example:"
    echo " Username: gameuser"
    echo " Database: humor_memory_game"
    echo " Password: Check your terraform.tfvars or set manually"
    RDS_USER="gameuser"
    RDS_DB="humor_memory_game"
    read -sp "Enter RDS password: " RDS_PASSWORD
    echo
fi

# Create postgres-secret
kubectl create secret generic postgres-secret \
    --from-literal=host=$RDS_HOST \
    --from-literal=port=$RDS_PORT \
    --from-literal=user=$RDS_USER \
    --from-literal=password=$RDS_PASSWORD \
    --from-literal=database=$RDS_DB \

--from-literal=url="postgresql://${RDS_USER}:${RDS_PASSWORD}@${RDS_HOST}:${RDS_PORT}/${RDS_DB}" \
    --namespace=h-game \
    --dry-run=client -o yaml | kubectl apply -f -

# Verify secret created
kubectl get secret postgres-secret -n h-game

# Expected output:

```

# NAME	TYPE	DATA	AGE
# postgres-secret	Opaque	6	5s

Get Redis credentials (if using Redis):

```
# Get Redis endpoint from Terraform
cd h-game-terraform
REDIS_ENDPOINT=$(terraform output -raw elasticache_endpoint)
REDIS_PORT=$(terraform output -raw elasticache_port)

echo "Redis Endpoint: $REDIS_ENDPOINT"
echo "Redis Port: $REDIS_PORT"

# Create redis-secret
kubectl create secret generic redis-secret \
  --from-literal=host=$REDIS_ENDPOINT \
  --from-literal=port=$REDIS_PORT \
  --from-literal=url="redis://${REDIS_ENDPOINT}:${REDIS_PORT}" \
  --namespace=h-game \
  --dry-run=client -o yaml | kubectl apply -f -

# Verify secret created
kubectl get secret redis-secret -n h-game
```

Alternative: If Terraform outputs don't work, create secrets manually:

```
# Create postgres-secret manually
kubectl create secret generic postgres-secret \
  --from-literal=host=your-rds-endpoint.region.rds.amazonaws.com \
  --from-literal=port=5432 \
  --from-literal=user=gameuser \
  --from-literal=password=YOUR_SECURE_PASSWORD \
  --from-literal=database=humor_memory_game \
  --from-literal=url="postgresql://gameuser:YOUR_SECURE_PASSWORD@your-rds-endpoint.region.rds.amazonaws.com:5432/humor_memory_game" \
  --namespace=h-game
```

```
# Create redis-secret manually
kubectl create secret generic redis-secret \
  --from-literal=host=your-redis-endpoint.cache.amazonaws.com \
  --from-literal=port=6379 \

--from-literal=url="redis://your-redis-endpoint.cache.amazonaws.com:6
379" \
  --namespace=h-game
```

Step 0.7: Verify ECR Images Exist (Optional but Recommended)

```
# Check if backend image exists in ECR
aws ecr describe-images \
  --repository-name h-game-backend \
  --region us-east-1 \
  --query 'imageDetails[*].imageTags[0]' \
  --output table

# Expected output:
# -----
# | DescribeImages |
# -----
# | latest        |
# | abc123def456   |
# -----

# If no images exist, you'll need to build and push:
# See Section 7 (CI/CD) or run:
# cd backend
# docker build -t h-game-backend .
# docker tag h-game-backend:latest
${ECR_REGISTRY}/h-game-backend:latest
# aws ecr get-login-password --region us-east-1 | docker login
--username AWS --password-stdin ${ECR_REGISTRY}
# docker push ${ECR_REGISTRY}/h-game-backend:latest
```

Step 0.8: Verify ECR Authentication (For Image Pulls)

Check if EKS nodes can pull from ECR:

```
# Check if EKS node group has ECR permissions
# This should be configured in Terraform, but verify:

# Get node group IAM role
NODE_ROLE=$(aws eks describe-nodegroup \
  --cluster-name $(cd h-game-terraform && terraform output -raw eks_cluster_name) \
  --nodegroup-name $(aws eks list-nodegroups --cluster-name $(cd h-game-terraform && terraform output -raw eks_cluster_name) --query 'nodegroups[0]' --output text) \
  --query 'nodegroup.nodeRole' --output text | cut -d'/' -f2)

echo "Node Group Role: $NODE_ROLE"

# Check if role has ECR read policy
aws iam list-attached-role-policies --role-name $NODE_ROLE | grep -i ecr

# Expected: Should show AmazonEC2ContainerRegistryReadOnly policy
```

If ECR permissions are missing, add them:

```
# Attach ECR read-only policy to node group role
aws iam attach-role-policy \
  --role-name $NODE_ROLE \
  --policy-arn
arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly
```

Alternative: Create imagePullSecret (if IRSA not configured):

```
# Create ECR authentication secret
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
ECR_REGISTRY="${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com"
```



```
kubectl create secret docker-registry ecr-secret \
  --docker-server=$ECR_REGISTRY \
  --docker-username=AWS \
  --docker-password=$(aws ecr get-login-password --region us-east-1) \
  --namespace=h-game \
  --dry-run=client -o yaml | kubectl apply -f -

# Note: You'll need to add imagePullSecrets to deployment manifests
# See troubleshooting section below
```

Step 0.9: Final Verification Checklist

Run this complete checklist:

```
echo "=== Prerequisites Checklist ==="
echo ""

# 1. EKS Cluster
echo -n "✓ EKS Cluster: "
aws eks list-clusters --region us-east-1 --query 'clusters[0]'
--output text && echo "OK" || echo "MISSING"

# 2. kubectl
echo -n "✓ kubectl: "
kubectl get nodes --no-headers 2>/dev/null | wc -l | xargs -I {} echo
"{} nodes" || echo "NOT CONFIGURED"

# 3. ECR Registry
echo -n "✓ ECR Registry: "
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output
text 2>/dev/null)
[ -n "$ACCOUNT_ID" ] && echo
"${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com" || echo "CANNOT GET"

# 4. ECR Repositories
echo -n "✓ ECR Backend Repo: "
```

```

aws ecr describe-repositories --repository-names h-game-backend
--region us-east-1 --query 'repositories[0].repositoryName' --output
text 2>/dev/null && echo "OK" || echo "MISSING"

echo -n "✓ ECR Frontend Repo: "
aws ecr describe-repositories --repository-names h-game-frontend
--region us-east-1 --query 'repositories[0].repositoryName' --output
text 2>/dev/null && echo "OK" || echo "MISSING"

# 5. Namespace
echo -n "✓ Namespace: "
kubectl get namespace h-game --no-headers 2>/dev/null && echo "OK" ||
echo "MISSING"

# 6. Secrets
echo -n "✓ postgres-secret: "
kubectl get secret postgres-secret -n h-game --no-headers 2>/dev/null
&& echo "OK" || echo "MISSING"

echo -n "✓ redis-secret: "
kubectl get secret redis-secret -n h-game --no-headers 2>/dev/null &&
echo "OK" || echo "MISSING"

echo ""
echo "=== Checklist Complete ==="
echo ""
echo "If any items show, fix them before proceeding."
echo "See troubleshooting section below for help."

```

Troubleshooting Prerequisites

Issue: "EKS cluster not found"

```
# Recreate infrastructure
cd h-game-terraform
terraform apply

# Wait for cluster to be ready (10-15 minutes)
aws eks wait cluster-active --name $(terraform output -raw
eks_cluster_name) --region us-east-1
```

Issue: "kubectl: command not found"

```
# Install kubectl
# Mac:
brew install kubectl

# Linux:
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

Issue: "Terraform outputs not found"

```
# Make sure you're in the right directory
cd h-game-terraform

# Initialize Terraform if needed
terraform init

# Check if outputs exist
terraform output

# If outputs are empty, infrastructure may not be created
terraform apply
```

Issue: "Secret creation fails"

```
# Check if namespace exists
kubectl get namespace h-game

# If not, create it
kubectl create namespace h-game

# Try creating secret again
# (Use commands from Step 0.6)
```

Issue: "ECR authentication fails"

```
# Verify AWS credentials
aws sts get-caller-identity

# Test ECR login
aws ecr get-login-password --region us-east-1 | docker login
--username AWS --password-stdin ${ECR_REGISTRY}

# If this works, ECR access is fine
# If it fails, check IAM permissions
```

Once all prerequisites pass, proceed to "Phase 1: Understanding GitOps"

Phase 1: Understanding GitOps (45 min)

What is GitOps?

GitOps is an operational framework where:

1. **Git as source of truth:** All configs in Git
2. **Declarative:** Describe desired state, not steps
3. **Automated:** Changes applied automatically
4. **Observable:** Git history = audit trail

Traditional CD vs GitOps:

Traditional CD (Section 7):

```
Developer → Push code → CI builds → CI deploys to K8s
```

- CI has kubectl access
- No single source of truth
- Hard to audit who deployed what

GitOps (Section 9):

```
Developer → Push code → CI builds → CI updates Git manifests
                                     ↓
ArgoCD monitors Git → Detects change → Deploys to K8s
```

- CI never touches K8s
- Git = source of truth
- Full audit trail in Git history

Why ArgoCD?

ArgoCD is the industry standard for GitOps:

Why pull-based model?

- **Security:** ArgoCD pulls from Git (no push access needed to cluster)
- **Control:** Cluster controls what it pulls (not external system pushing)
- **Reliability:** Works even if CI/CD is down (ArgoCD keeps monitoring Git)
- **Compliance:** Easier to audit (cluster-initiated actions)

Why the UI?

- **Visibility:** See all applications and their status at a glance
- **Debugging:** Visual resource tree shows what's deployed
- **Onboarding:** New team members can understand deployments without CLI
- **Monitoring:** Real-time sync status and health checks

Why automatic drift detection?

- **Self-healing:** Corrects manual changes automatically
- **Compliance:** Ensures cluster always matches Git (no configuration drift)
- **Reliability:** Prevents "works on my machine" issues (Git is source of truth)

Why multi-cluster support?

- **Scalability:** Manage dev, staging, prod clusters from one ArgoCD
- **Consistency:** Same Git repo deploys to all clusters
- **Efficiency:** One tool for all environments

Why RBAC integration?

- **Security:** Control who can deploy what
- **Team isolation:** Different teams manage different applications
- **Audit:** Track who made what changes

Why free and open-source?

- **Cost:** No licensing fees (important for learning and startups)
- **Transparency:** Can inspect and modify code
- **Community:** Large ecosystem and support

Key Concepts:

- **Application:** ArgoCD resource representing deployed app
- **Repository:** Git repo containing manifests
- **Sync:** Process of applying Git state to cluster
- **Drift:** When cluster state differs from Git
- **Self-healing:** Automatic correction of drift

Design Exercise 1: GitOps Repository Structure

Scenario: You need to organize H-Game manifests for GitOps.

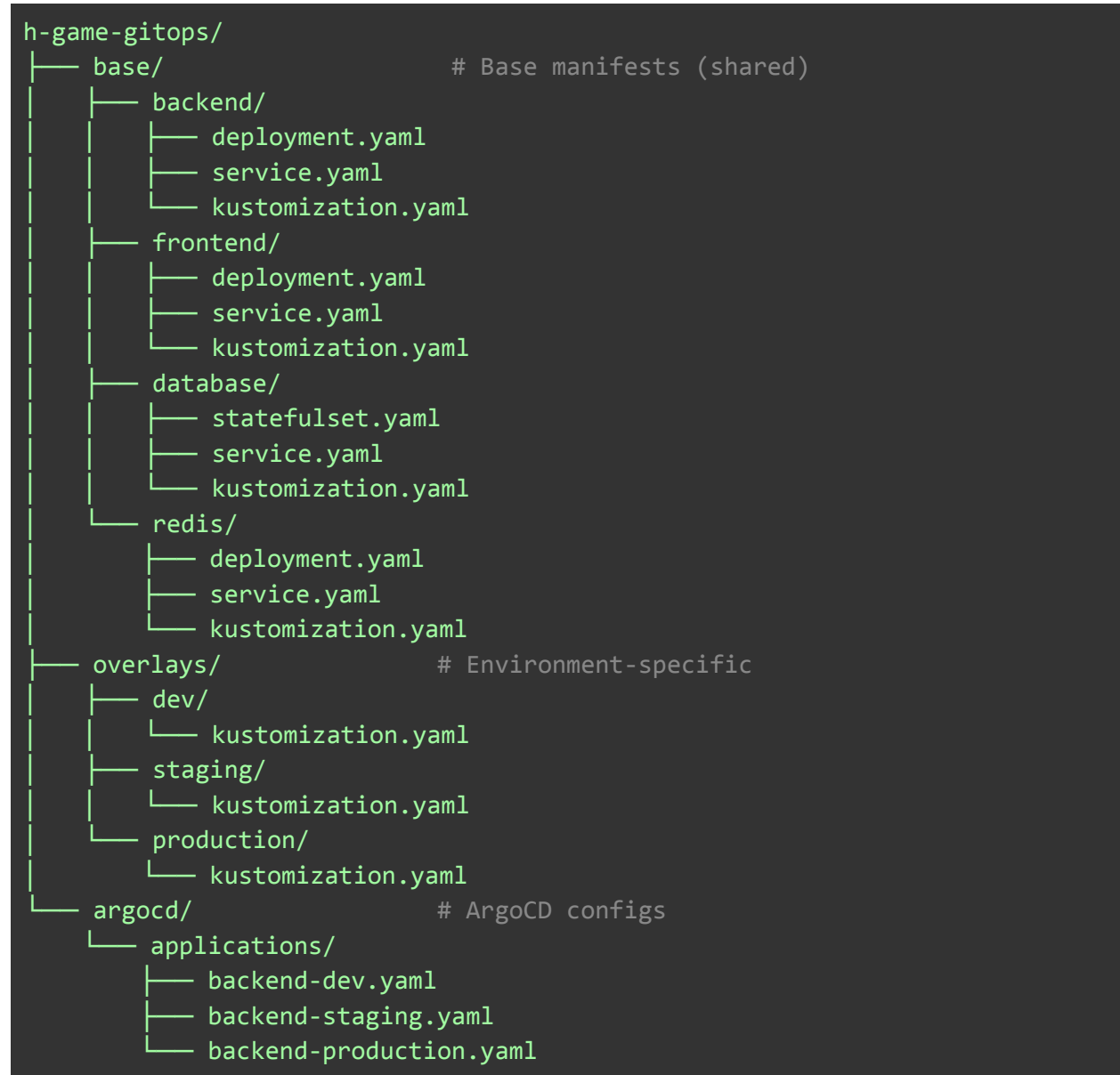
Question: How should you structure your GitOps repository?

Think about:

- Base manifests (shared across environments)
- Environment-specific configs (dev/staging/prod)
- ArgoCD Application definitions
- Secrets management

Your answer: _____

Answer Below 

Answer:**Recommended GitOps Repository Structure:****Why this structure:****Why base/ directory?**

- DRY principle: Don't repeat yourself - shared configs in one place
- Maintainability: Change base once, affects all environments

- Consistency: Ensures all environments use same base configuration
- Simplicity: Clear what's shared vs what's environment-specific

Why overlays/ directory?

- Environment isolation: Each environment has its own config
- Flexibility: Easy to customize per environment (replicas, resources, image tags)
- Safety: Can't accidentally change production when editing dev
- Clarity: Immediately see what differs between environments

Why argocd/ directory?

- Separation: ArgoCD configs separate from application configs
- Organization: Easy to find all Application definitions
- Versioning: Can version ArgoCD configs independently
- Best practice: Industry-standard GitOps repository structure

Design Exercise 2: What Goes in Git?

Scenario: You're setting up GitOps for H-Game.

Question: What should be committed to Git?

Think about:

- Kubernetes manifests? (YES/NO)
- Helm charts? (YES/NO)
- Secrets? (YES/NO)
- Generated files? (YES/NO)
- Infrastructure code? (YES/NO)

Your answer: _____

Answer Below 

Answer:**What Goes in Git:**

- **Kubernetes manifests:** YES (Deployments, Services, ConfigMaps)
- **Helm charts:** YES (if using Helm)
- **Kustomize configs:** YES (base and overlays)
- **ArgoCD Applications:** YES (Application definitions)
- **Secrets:** NO (use External Secrets Operator or Sealed Secrets)
- **Generated files:** NO (build artifacts, compiled code)
- **Infrastructure code:** Separate repo (Terraform in different repo)

Security Note: Never commit secrets to Git. Use External Secrets Operator (from Section 6) to sync secrets from AWS Secrets Manager.

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is GitOps?	"GitOps uses Git as the single source of truth for infrastructure. ArgoCD monitors Git and deploys changes automatically."
Why use ArgoCD?	"ArgoCD provides a UI, automatic drift detection, and self-healing capabilities for GitOps deployments."
What is drift?	"Drift is when the cluster state differs from Git. ArgoCD detects and corrects drift automatically."
What is an Application?	"An Application is an ArgoCD resource that defines what to deploy (repo, path, cluster, namespace)."

If you can explain these concepts, you're ready for implementation.

Phase 2: Implementation (2.5 hours)

Step 1: Install ArgoCD

Time: 15 min

Create namespace:

```
kubectl create namespace argocd
```

Install ArgoCD:

```
kubectl apply -n argocd -f  
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/i  
ninstall.yaml
```

Expected output:

```
namespace/argocd created  
customresourcedefinition.apiextensions.k8s.io/applications.argoproj.i  
o created  
customresourcedefinition.apiextensions.k8s.io/applicationsets.argopro  
j.io created  
...  
deployment.apps/argocd-server created  
deployment.apps/argocd-repo-server created  
deployment.apps/argocd-application-controller created
```

Wait for pods to be ready:

```
kubectl wait --for=condition=ready pod -l  
app.kubernetes.io/name=argocd-server -n argocd --timeout=300s
```

Verify installation:

```
kubectl get pods -n argocd
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
argocd-application-controller-0	1/1	Running	0	2m
argocd-applicationset-controller-7d8b9c4b6-abc12	1/1	Running	0	2m
argocd-dex-server-5f8d9c4b6-def34	1/1	Running	0	2m
argocd-notifications-controller-9c4b6def-ghi56	1/1	Running	0	2m
argocd-redis-7d4b8c9f5-jkl78	1/1	Running	0	2m
argocd-repo-server-5f8d9c4b6-mno90	1/1	Running	0	2m
argocd-server-7d4b8c9f5-pqr12	1/1	Running	0	2m

All pods should be Running. Wait 2-3 minutes if some are still starting.

Step 2: Access ArgoCD UI

Time: 10 min

Port-forward ArgoCD server:

```
kubect1 port-forward svc/argocd-server -n argocd 8080:443
```

Get initial admin password:

```
kubect1 -n argocd get secret argocd-initial-admin-secret -o  
jsonpath="{.data.password}" | base64 -d && echo
```

Expected output:

```
a1b2c3d4e5f6g7h8i9j0 # Your password (will be different)
```

Access ArgoCD UI:

- Open browser: <https://localhost:8080>
- Accept self-signed certificate warning
- Username: **admin**
- Password: (from above command)

Expected result:

- ArgoCD dashboard loads
- Empty application list (no applications yet)
- Settings menu accessible

Change password (recommended):

1. Click user icon (top right) → Account Settings
2. Click "Update Password"
3. Set new password
4. Save

Step 3: Install ArgoCD CLI**Time: 10 min****Mac:**

```
brew install argocd
```

Linux:

```
curl -sSL -o argocd  
https://github.com/argoproj/argo-cd/releases/latest/download/argocd-l  
inux-amd64  
chmod +x argocd  
sudo mv argocd /usr/local/bin/
```

Windows:

```
choco install argocd-cli
```

Or download from: <https://github.com/argoproj/argo-cd/releases/latest>**Verify installation:**

```
argocd version --client
```

Expected output:

```
argocd: v2.10.0+abc123
BuildDate: 2024-01-15T10:00:00Z
GitCommit: abc123def456
```

Login via CLI:

```
argocd login localhost:8080 --username admin --password
<your-password> --insecure
```

Expected output:

```
'admin' logged in successfully
Context 'localhost:8080' updated
```

Step 4: Create GitOps Repository**Time: 20 min****Create new GitHub repository:**

1. Go to GitHub → New repository
2. Repository name: h-game-gitops
3. Description: GitOps repository for H-Game application
4. Public or Private: Your choice (public recommended for learning)
5. Initialize: Check "Add a README"
6. Click "Create repository"

Clone locally:

```
cd ~/projects
git clone https://github.com/YOUR_USERNAME/h-game-gitops.git
cd h-game-gitops
```


Create directory structure:

```
mkdir -p base/backend base/frontend base/database base/redis  
mkdir -p overlays/dev overlays/staging overlays/production  
mkdir -p argocd/applications
```

Create README:

```
cat > README.md << 'EOF'  
# H-Game GitOps Repository  
  
This repository contains Kubernetes manifests for H-Game application  
managed by ArgoCD.  
  
## Structure  
  
- base/ - Base manifests shared across environments  
- overlays/ - Environment-specific configurations  
- argocd/ - ArgoCD Application definitions  
  
## Environments  
  
- `dev` - Development environment  
- `staging` - Staging environment  
- `production` - Production environment  
  
## Usage  
  
ArgoCD monitors this repository and automatically syncs changes to  
Kubernetes clusters.  
EOF
```

Initial commit:

```
git add .  
git commit -m "feat: Initialize GitOps repository structure"  
git push origin main
```

Step 5: Move Manifests to Git

Time: 30 min

Copy backend manifests:

```
# If you have existing manifests in a k8s/ directory
cp ~/projects/H-Game/k8s/backend-deployment.yaml
base/backend/deployment.yaml
cp ~/projects/H-Game/k8s/backend-service.yaml
base/backend/service.yaml

# Or create new manifests
```

Create backend deployment:

"Use the ECR_REGISTRY value from Step 0.3"

```
# base/backend/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: h-game
  labels:
    app: backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: ${ECR_REGISTRY}h-game-backend:latest
          ports:
            - containerPort: 3001
```

```

    name: http
  env:
    - name: NODE_ENV
      value: "production"
    - name: DATABASE_URL
      valueFrom:
        secretKeyRef:
          name: postgres-secret
          key: url
    - name: REDIS_URL
      valueFrom:
        secretKeyRef:
          name: redis-secret
          key: url
  resources:
    requests:
      cpu: 200m
      memory: 256Mi
    limits:
      cpu: 500m
      memory: 512Mi
  livenessProbe:
    httpGet:
      path: /health
      port: 3001
    initialDelaySeconds: 30
    periodSeconds: 10
  readinessProbe:
    httpGet:
      path: /health
      port: 3001
    initialDelaySeconds: 10
    periodSeconds: 5

```

Create backend service:

```

# base/backend/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: backend

```

```
namespace: h-game
labels:
  app: backend
spec:
  selector:
    app: backend
  ports:
  - port: 3001
    targetPort: 3001
    name: http
  type: ClusterIP
```

Create Kustomization:

```
# base/backend/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game

resources:
- deployment.yaml
- service.yaml
```

Commit to Git:

```
git add base/
git commit -m "feat: Add backend base manifests"
git push origin main
```

Step 6: Create ArgoCD Application

Ensure secrets exist (created in Step 0.6)

Time: 20 min

Create application manifest:

```
# argocd/applications/backend-dev.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: h-game-backend-dev
  namespace: argocd
spec:
  project: default

  source:
    repoURL: https://github.com/YOUR_USERNAME/h-game-gitops.git
    targetRevision: main
    path: base/backend

  destination:
    server: https://kubernetes.default.svc
    namespace: h-game

  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Replace **YOUR_USERNAME** with your GitHub username.

Apply application:

```
kubectl apply -f argocd/applications/backend-dev.yaml
```

Expected output:

```
application.argoproj.io/h-game-backend-dev created
```

Verify in ArgoCD UI:

1. Refresh ArgoCD UI (<https://localhost:8080>)
2. Application should appear in list
3. Status: "Syncing" → "Healthy" and "Synced"

Or verify via CLI:

```
argocd app get h-game-backend-dev
```

Expected output:

```
Name:          h-game-backend-dev
Project:       default
Server:       https://kubernetes.default.svc
Namespace:    h-game
Repo:         https://github.com/YOUR_USERNAME/h-game-gitops.git
Target:       main
Path:         base/backend
Sync Policy:   Automated (Prune: true, SelfHeal: true)
Sync Status:   Synced to main (abc123)
Health Status: Healthy
```

Step 7: Verify Deployment

Time: 15 min

Check pods:

```
kubectl get pods -n h-game -l app=backend
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
backend-7d4b8c9f5-abc12	1/1	Running	0	2m
backend-7d4b8c9f5-def34	1/1	Running	0	2m
backend-7d4b8c9f5-ghi56	1/1	Running	0	2m

Check service:

```
kubectl get svc -n h-game backend
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
backend	ClusterIP	10.100.50.123	<none>	3001/TCP	5m

View application in ArgoCD UI:

1. Click on [h-game-backend-dev](#) application
2. View resource tree
3. See Deployment, Service, Pods
4. All should show green (Healthy)

Phase 3: Validation (30 min)

Test GitOps Flow

Test 1: Git Change Triggers Deployment

```
# 1. Edit deployment in Git
cd ~/projects/h-game-gitops
vim base/backend/deployment.yaml
# Change replicas: 3 → 4

# 2. Commit and push
git add .
git commit -m "scale: Increase backend replicas to 4"
git push origin main

# 3. Watch ArgoCD sync (automatic, within 3 minutes)
# In ArgoCD UI, application should show "OutOfSync" → "Syncing" → "Synced"

# Or via CLI:
argocd app wait h-game-backend-dev

# 4. Verify in cluster
kubectl get pods -n h-game -l app=backend
# Should show 4 backend pods
```

Expected result:

- ArgoCD detects Git change within 3 minutes
- Application syncs automatically
- Cluster has 4 backend pods

Test 2: Drift Detection and Self-Healing

```
# 1. Manually change cluster (create drift)
kubectl scale deployment backend --replicas=5 -n h-game

# 2. Watch ArgoCD detect drift (within 3 minutes)
argocd app get h-game-backend-dev
# Should show "OutOfSync" status

# 3. Wait for self-healing (automatic with selfHeal: true)
argocd app wait h-game-backend-dev

# 4. Verify self-healing
kubectl get pods -n h-game -l app=backend
# Should be back to 4 replicas (Git state)
```

Expected result:

- ArgoCD detects drift (cluster has 5, Git says 4)
- ArgoCD corrects drift automatically
- Cluster matches Git state (4 replicas)

Test 3: Rollback via Git

```
# 1. View Git history
cd ~/projects/h-game-gitops
git log --oneline

# 2. Rollback to previous commit
git revert HEAD
git push origin main

# 3. Watch ArgoCD sync
argocd app wait h-game-backend-dev

# 4. Verify rollback
kubectl get pods -n h-game -l app=backend
# Should be back to 3 replicas (previous state)
```

Expected result:

- Git revert creates new commit
- ArgoCD syncs to previous state
- Cluster matches previous Git state

Common Errors & Fixes

Error: ArgoCD pods not starting

```
# Check pod status
kubectl get pods -n argocd

# Check logs
kubectl logs -n argocd deployment/argocd-server

# Common fix: Wait longer (ArgoCD takes 2-3 minutes to start)
kubectl wait --for=condition=ready pod -l
app.kubernetes.io/name=argocd-server -n argocd --timeout=600s
```

Error: Application shows "Unknown" or "Degraded"

```
# Check application events
argocd app get h-game-backend-dev --show-events

# Common causes:
# - Image pull errors (check ECR permissions)
# - Resource limits too low
# - Health checks failing

# Fix: Check pod logs
kubectl logs -n h-game deployment/backend
```

Error: ArgoCD can't access Git repository

```
# For public repos: No authentication needed
# For private repos: Add repository credentials in ArgoCD UI

# Steps:
# 1. ArgoCD UI → Settings → Repositories
# 2. Click "Connect Repo"
# 3. Enter repository URL
# 4. Add credentials (username/password or SSH key)
```

Error: Sync fails with "permission denied"

```
# Check RBAC permissions
kubectl auth can-i create deployments --namespace=h-game

# Fix: Ensure ArgoCD has proper RBAC
# ArgoCD install includes necessary RBAC by default
```

Key Takeaways

- ArgoCD continuously monitors Git for changes
- Cluster state automatically syncs to Git state
- Drift is detected and corrected (self-healing)
- Full audit trail in Git history
- Rollback is simple (Git revert)

Bridge to G2

ArgoCD is installed and basic sync works. Now let's deploy your complete H-Game application (backend, frontend, database, Redis) with proper dependency management and health checks...

G2: Application Deployment with ArgoCD

Time: 3-4 hours | **Cost:** Free

Mission Brief

Deploy your complete H-Game application (backend, frontend, database, Redis) using ArgoCD with proper dependency management and health checks. Learn how to structure multiple ArgoCD Applications and manage application dependencies.

Outcomes:

- Deploy complete H-Game application via ArgoCD
- Create multiple ArgoCD Applications
- Configure health checks and sync policies
- Understand application dependencies
- Monitor deployment status in ArgoCD UI

Prerequisites:

- **G1 complete** - ArgoCD basics understood
- **EKS cluster created** - Via Terraform (T3: EKS Cluster)
- **ArgoCD installed** - On your EKS cluster
- **Docker images built and pushed** - Backend image in ECR (from CI/CD or manual build)
- **GitOps repository created** - Separate repo for Kubernetes manifests
- **kubectl configured** - Can access your EKS cluster

Note: You don't need the app already deployed to EKS. G2 is the step that deploys it.

Before You Start: Prerequisites Checklist

Important: G2 is the step that deploys your app to EKS. You don't need the app already running - this tutorial will deploy it.

EKS cluster exists and is accessible

```
# Verify cluster exists
aws eks list-clusters

# Verify kubectl can access cluster
```

```
kubectl cluster-info
kubectl get nodes
# Should show your EKS worker nodes
```

ArgoCD installed and running on EKS

```
# Verify ArgoCD is installed
kubectl get pods -n argocd
# Should show: argocd-server, argocd-repo-server,
argocd-application-controller

# If ArgoCD is not installed, install it:
kubectl create namespace argocd
kubectl apply -n argocd -f
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/i
ninstall.yaml
# Wait for pods to be ready (may take 2-3 minutes)
kubectl wait --for=condition=ready pod -l
app.kubernetes.io/name=argocd-server -n argocd --timeout=300s
```

ArgoCD CLI installed

```
# macOS
brew install argocd

# Linux
curl -sSL -o /usr/local/bin/argocd
https://github.com/argoproj/argo-cd/releases/latest/download/argocd-l
inux-amd64
chmod +x /usr/local/bin/argocd

# Verify installation
argocd version --client
```

ArgoCD CLI configured

```
# Get ArgoCD admin password
ARGOCD_PASSWORD=$(kubectl -n argocd get secret
argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64
-d)

# Port-forward ArgoCD server (in separate terminal)
kubectl port-forward svc/argocd-server -n argocd 8080:443

# Login (in another terminal)
argocd login localhost:8080 --username admin --password
$ARGOCD_PASSWORD --insecure
```

Docker images built and pushed to ECR

```
# Verify backend image exists in ECR
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output
text)
ECR_REGISTRY="${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com"

# List images in ECR
aws ecr list-images --repository-name h-game-backend --region
us-east-1

# If image doesn't exist, build and push:
# cd backend
# docker build -t h-game-backend:latest .
# aws ecr get-login-password --region us-east-1 | docker login
--username AWS --password-stdin $ECR_REGISTRY
# docker tag h-game-backend:latest
$ECR_REGISTRY/h-game-backend:latest
# docker push $ECR_REGISTRY/h-game-backend:latest
```

GitOps repository created and accessible

```
# Create GitOps repo if it doesn't exist
# mkdir -p ~/projects/h-game-gitops
# cd ~/projects/h-game-gitops
# git init
# git remote add origin
https://github.com/YOUR_USERNAME/h-game-gitops.git

# Verify you can push to your GitOps repo
cd ~/projects/h-game-gitops
git remote -v
```

ECR registry configured

```
# Get your ECR registry
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
ECR_REGISTRY="${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com"
echo "ECR Registry: $ECR_REGISTRY"
```

The One Thing That Makes This Click

ArgoCD Application = blueprint for deployment. Just like a blueprint describes how to build a house (foundation first, then walls, then roof), an ArgoCD Application describes how to deploy your app (what repo, what path, what cluster, what namespace).

Without Applications:

- Manual deployment of each component
- No visibility into deployment status
- Hard to track what's deployed where
- No dependency management

With Applications:

- Each component is an Application
- ArgoCD UI shows deployment status
- Clear visibility into what's deployed
- Dependencies managed via sync waves

In essence: Applications are the deployment units. Everything else (sync policies, health checks, dependencies) is just configuration.

The Mental Model Shift:

BEFORE G2:

- "I deploy backend, frontend, database separately"
- "I hope they start in the right order"
- "No visibility into deployment status"

AFTER G2:

- "Each component is an Application"
- "Sync waves control deployment order"
- "ArgoCD UI shows real-time status"

This is the difference between "manual deployment" and "declarative deployment."

Now let's see how application deployment works...

Phase 1: Understanding Applications (30 min)

ArgoCD Application Structure

An Application defines:

- **Source:** Where manifests are (Git repo + path)
- **Destination:** Where to deploy (cluster + namespace)
- **Sync Policy:** How to sync (auto/manual, prune, self-heal)

Application Example:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: h-game-backend
  namespace: argocd
spec:
  source:      # Git repo + path
  destination: # Cluster + namespace
  syncPolicy:  # Auto-sync, prune, self-heal
```

Sync Policies

Manual Sync:

- Require manual trigger (`argocd app sync`)
- Good for production (controlled deployments)

Why manual sync for production?

- **Control:** Require explicit approval before deployment
- **Safety:** Prevent accidental deployments from Git commits
- **Review:** Team can review changes before applying
- **Compliance:** Some organizations require manual approval for production

Automated Sync:

- Auto-sync on Git change
- Good for dev/staging (fast feedback)

Why automated sync for dev/staging?

- **Speed:** Fast feedback loop (see changes immediately)
- **Iteration:** Developers can test changes quickly
- **Efficiency:** No manual intervention needed for non-production
- **Learning:** See GitOps in action automatically

Prune:

- Delete resources not in Git
- Keeps cluster clean

Why prune?

- **Cleanliness:** Remove resources deleted from Git
- **Consistency:** Cluster exactly matches Git (no orphaned resources)
- **Cost:** Prevents unused resources from consuming resources
- **Safety:** Can disable for production if needed (`prune: false`)

Self-heal:

- Revert manual cluster changes
- Maintains Git as source of truth

Why self-heal?

- **Compliance:** Ensures cluster always matches Git (no configuration drift)
- **Reliability:** Prevents manual changes from causing issues
- **Audit:** All changes must go through Git (full audit trail)
- **Team discipline:** Enforces GitOps workflow (no manual kubectl)

Health Checks

ArgoCD monitors:

- Deployment: Ready replicas vs desired
- Service: Endpoints available
- Pod: Running and ready
- Custom: Health check scripts

Design Exercise: Application Dependencies

Scenario: You need to deploy H-Game with proper startup order.

Question: What's the correct deployment order?

Think about:

- Database must be ready first
- Redis must be ready second
- Backend depends on DB and Redis
- Frontend depends on Backend

Your answer: _____

Answer:

Deployment Order:

1. **Database (PostgreSQL)** - Must be ready first
2. **Redis** - Must be ready second
3. **Backend** - Depends on DB and Redis
4. **Frontend** - Depends on Backend

Implementation:

- Use sync waves (G4) for ordered deployment
- Or deploy sequentially (database → redis → backend → frontend)
- Health checks ensure readiness before next component

Phase 2: Implementation (2.5 hours)

Step 1: Structure Base Manifests

Create Database Manifests

IMPORTANT: Secret Management Options

You have two options for managing secrets:

Option A: Plain Kubernetes Secret (For Learning/No Infrastructure)

- Use this if you don't have Terraform infrastructure yet
- Simple, works immediately
- Not recommended for production

Option B: AWS Secrets Manager (If You Have Terraform Infrastructure)

- Use this if you've completed Terraform setup (T9: Secrets Manager)
- Secrets stored in AWS Secrets Manager (encrypted, managed by Terraform)
- More secure, production-ready
- Requires External Secrets Operator or manual sync

Option A: Plain Kubernetes Secret (Default for Learning)

File: **base/database/secret.yaml**

```
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
  namespace: h-game
type: Opaque
stringData:
  postgres-db: hgame
  postgres-user: postgres
  postgres-password: postgres123 # For learning only - use AWS
  Secrets Manager in production
```

Option B: AWS Secrets Manager (If You Have Terraform Infrastructure)

If you have Terraform infrastructure with Secrets Manager (T9), your secrets are stored in AWS Secrets Manager with the name:

- Secret name: `h-game/rds/password` (or `${var.project_name}/rds/password`)
- Contains: `username`, `password`, `engine`, `host`, `port`, `dbname`

Method 1: Sync AWS Secrets Manager to Kubernetes Secret (Manual)

Create a Kubernetes Secret that syncs from AWS Secrets Manager:

File: `base/database/secret-external.yaml` (alternative to `secret.yaml`)

```
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
  namespace: h-game
  annotations:
    # Note: This requires External Secrets Operator installed
    # For manual sync, use kubectl command below instead
type: Opaque
stringData:
  # These will be populated from AWS Secrets Manager
  postgres-db: "" # Will be synced from AWS
  postgres-user: "" # Will be synced from AWS
  postgres-password: "" # Will be synced from AWS
```

Manual sync command (run once, or via CI/CD):

```
# Get secret from AWS Secrets Manager
SECRET_JSON=$(aws secretsmanager get-secret-value \
  --secret-id h-game/rds/password \
  --query SecretString \
  --output text)

# Extract values
DB_USER=$(echo $SECRET_JSON | jq -r '.username')
```

```

DB_PASSWORD=$(echo $SECRET_JSON | jq -r '.password')
DB_NAME=$(echo $SECRET_JSON | jq -r '.dbname')

# Create Kubernetes Secret
kubectl create secret generic postgres-secret \
  --from-literal=postgres-user="$DB_USER" \
  --from-literal=postgres-password="$DB_PASSWORD" \
  --from-literal=postgres-db="$DB_NAME" \
  --namespace=h-game \
  --dry-run=client -o yaml | kubectl apply -f -

```

Method 2: External Secrets Operator (Recommended for Production)

Install External Secrets Operator to automatically sync AWS Secrets Manager to Kubernetes:

```

# Install External Secrets Operator
helm repo add external-secrets https://charts.external-secrets.io
helm install external-secrets \
  external-secrets/external-secrets \
  -n external-secrets-system \
  --create-namespace

# Create ExternalSecret resource (references AWS Secrets Manager)

```

File: `base/database/external-secret.yaml` (instead of `secret.yaml`)

```

apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: postgres-secret
  namespace: h-game
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: SecretStore

```



```
target:
  name: postgres-secret
  creationPolicy: Owner
data:
- secretKey: postgres-user
  remoteRef:
    key: h-game/rds/password
    property: username
- secretKey: postgres-password
  remoteRef:
    key: h-game/rds/password
    property: password
- secretKey: postgres-db
  remoteRef:
    key: h-game/rds/password
    property: dbname
```

Which option to use?

- **Learning/No Infrastructure:** Use Option A (plain Kubernetes Secret)
- **Already have Terraform Infrastructure:** Use Option B (AWS Secrets Manager)
 - **Quick setup:** Method 1 (manual sync)
 - **Production:** Method 2 (External Secrets Operator)

For this tutorial, we'll use Option A (plain Kubernetes Secret) for simplicity.

Create database manifests:

File: `base/database/statefulset.yaml`

```
# base/database/statefulset.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: h-game
  labels:
    app: postgres
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          env:
            - name: POSTGRES_DB
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: postgres-db
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: postgres-user
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: postgres-password
```

```

    ports:
      - containerPort: 5432
        name: postgres
    volumeMounts:
      - name: postgres-data
        mountPath: /var/lib/postgresql/data
    resources:
      requests:
        cpu: 200m
        memory: 256Mi
      limits:
        cpu: 500m
        memory: 512Mi
    volumeClaimTemplates:
      - metadata:
          name: postgres-data
        spec:
          accessModes: [ "ReadWriteOnce" ]
          resources:
            requests:
              storage: 10Gi

```

File: base/database/service.yaml

```

# base/database/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres
  namespace: h-game
  labels:
    app: postgres
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
      name: postgres
  type: ClusterIP

```

File: base/database/kustomization.yaml

```
# base/database/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game

resources:
- secret.yaml          # Use this if using Option A (plain Kubernetes Secret)
# - external-secret.yaml # Uncomment this if using Option B with External
# Secrets Operator
- statefulset.yaml
- service.yaml
```

Create Redis manifests:

File: base/redis/deployment.yaml

```
# base/redis/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: h-game
  labels:
    app: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
```

```

    image: redis:7
    ports:
      - containerPort: 6379
        name: redis
    resources:
      requests:
        cpu: 100m
        memory: 128Mi
      limits:
        cpu: 200m
        memory: 256Mi

```

File: base/redis/service.yaml

```

# base/redis/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: redis
  namespace: h-game
  labels:
    app: redis
spec:
  selector:
    app: redis
  ports:
    - port: 6379
      targetPort: 6379
      name: redis
  type: ClusterIP

```

File: base/redis/kustomization.yaml

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game

```

```
resources:  
- deployment.yaml  
- service.yaml
```

Create Backend Manifests

File: `base/backend/deployment.yaml`

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: backend  
  namespace: h-game  
  labels:  
    app: backend  
spec:  
  replicas: 3  
  selector:  
    matchLabels:  
      app: backend  
  template:  
    metadata:  
      labels:  
        app: backend  
    spec:  
      containers:  
        - name: backend  
          image: YOUR_ECR_REGISTRY/h-game-backend:latest  
          ports:  
            - containerPort: 3001  
              name: http  
          env:  
            - name: DB_HOST  
              value: postgres  
            - name: DB_PORT  
              value: "5432"
```

```
- name: DB_NAME
  valueFrom:
    secretKeyRef:
      name: postgres-secret
      key: postgres-db
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: postgres-secret
      key: postgres-user
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: postgres-secret
      key: postgres-password
- name: REDIS_HOST
  value: redis
- name: REDIS_PORT
  value: "6379"
resources:
  requests:
    cpu: 200m
    memory: 256Mi
  limits:
    cpu: 500m
    memory: 512Mi
livenessProbe:
  httpGet:
    path: /health
    port: 3001
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /health
    port: 3001
  initialDelaySeconds: 10
  periodSeconds: 5
```

File: base/backend/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: h-game
  labels:
    app: backend
spec:
  selector:
    app: backend
  ports:
    - port: 3001
      targetPort: 3001
      name: http
  type: ClusterIP
```

File: base/backend/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game

resources:
- deployment.yaml
- service.yaml
```


Create Frontend Manifests

Note: For learning purposes, we'll use `nginx:alpine` as a placeholder. Replace with your actual frontend image when ready.

Create frontend manifests:

```
# base/frontend/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: h-game
  labels:
    app: frontend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: YOUR_ECR_REGISTRY/h-game-frontend:latest
          ports:
            - containerPort: 80
              name: http
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 200m
              memory: 128Mi
          livenessProbe:
```

```

    httpGet:
      path: /health
      port: 80
      initialDelaySeconds: 10
      periodSeconds: 10
  readinessProbe:
    httpGet:
      path: /health
      port: 80
      initialDelaySeconds: 5
      periodSeconds: 5

```

File: base/frontend/service.yaml

```

# base/frontend/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: h-game
  labels:
    app: frontend
spec:
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
      name: http
      type: ClusterIP

```

File: base/frontend/kustomization.yaml

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game

```

```
resources:  
- deployment.yaml  
- service.yaml
```

Commit to Git:

```
cd ~/projects/h-game-gitops  
git add base/  
git commit -m "feat: Add database, redis, and frontend base  
manifests"  
git push origin main
```

Validation:

```
# Verify files are committed  
git log --oneline -1  
git ls-files base/
```

Step 2: Create All Applications (30 min)

Database Application

File: `argocd/applications/database.yaml`

```
apiVersion: argoproj.io/v1alpha1  
kind: Application  
metadata:  
  name: h-game-database  
  namespace: argocd  
spec:  
  project: default  
  source:  
    repoURL: https://github.com/YOUR_USERNAME/h-game-gitops.git  
    targetRevision: main  
    path: base/database
```

```

destination:
  server: https://kubernetes.default.svc
  namespace: h-game
syncPolicy:
  automated:
    prune: true
    selfHeal: true
  syncOptions:
    - CreateNamespace=true

```

Redis Application

File: `argocd/applications/redis.yaml`

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: h-game-redis
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/YOUR_USERNAME/h-game-gitops.git
    targetRevision: main
    path: base/redis
  destination:
    server: https://kubernetes.default.svc
    namespace: h-game
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true

```

Backend Application

File: `argocd/applications/backend.yaml`

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: h-game-backend
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/YOUR_USERNAME/h-game-gitops.git
    targetRevision: main
    path: base/backend
  destination:
    server: https://kubernetes.default.svc
    namespace: h-game
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Frontend Application

File: `argocd/applications/frontend.yaml`

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: h-game-frontend
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/YOUR_USERNAME/h-game-gitops.git
```

```

    targetRevision: main
    path: base/frontend
  destination:
    server: https://kubernetes.default.svc
    namespace: h-game
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true

```

Commit and Apply

```

cd ~/projects/h-game-gitops
git add argocd/
git commit -m "feat: Add all H-Game applications"
git push origin main

# Apply all applications
kubectl apply -f argocd/applications/database.yaml
kubectl apply -f argocd/applications/redis.yaml
kubectl apply -f argocd/applications/backend.yaml
kubectl apply -f argocd/applications/frontend.yaml

```

Validation:

```

# Verify applications created
kubectl get applications -n argocd
# Should show: h-game-database, h-game-redis, h-game-backend,
h-game-frontend

```

Step 3: Monitor Deployment (20 min)

View All Applications

```
argocd app list
```

Expected output:

NAME	HEALTH	SYNC STATUS
h-game-backend	Healthy	Synced
h-game-database	Healthy	Synced
h-game-frontend	Healthy	Synced
h-game-redis	Healthy	Synced

View Application Details

```
argocd app get h-game-backend
```

View in ArgoCD UI

1. Port-forward ArgoCD server (if not already running):

```
kubect1 port-forward svc/argocd-server -n argocd 8080:443
```

2. Open browser: <https://localhost:8080>
3. Login with username `admin` and password from earlier
4. View all applications
5. Click on each application to see resource tree
6. All should show green (Healthy and Synced)

Check Pods

```
kubect1 get pods -n h-game
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
backend-7d4b8c9f5-abc12	1/1	Running	0	5m
backend-7d4b8c9f5-def34	1/1	Running	0	5m
backend-7d4b8c9f5-ghi56	1/1	Running	0	5m
frontend-5f8d9c4b6-jkl78	1/1	Running	0	4m
frontend-5f8d9c4b6-mno90	1/1	Running	0	4m
frontend-5f8d9c4b6-pqr12	1/1	Running	0	4m
postgres-0	1/1	Running	0	6m
redis-9c4b6def-xyz34	1/1	Running	0	5m

Phase 3: Validation (30 min)**Test Complete Flow****Test 1: Update Backend Image**

```
# 1. Update image tag in Git
cd ~/projects/h-game-gitops
vim base/backend/deployment.yaml
# Change image: latest → v1.2.3

# 2. Commit and push
git commit -am "chore: Update backend image to v1.2.3"
git push origin main

# 3. Watch ArgoCD sync
argocd app wait h-game-backend

# 4. Verify deployment
kubectl get deployment backend -n h-game -o
jsonpath='{.spec.template.spec.containers[0].image}'
# Should show: YOUR_ECR_REGISTRY/h-game-backend:v1.2.3
```


Test 2: Scale Frontend

```
# 1. Update replicas in Git
vim base/frontend/deployment.yaml
# Change replicas: 3 → 5

# 2. Commit and push
git commit -am "scale: Increase frontend replicas to 5"
git push origin main

# 3. Watch ArgoCD sync
argocd app wait h-game-frontend

# 4. Verify scaling
kubectl get pods -n h-game -l app=frontend
# Should show 5 frontend pods
```

Test 3: Delete Resource from Git

```
# 1. Remove a resource from Git
rm base/redis/deployment.yaml
# Update kustomization.yaml to remove deployment.yaml reference

# 2. Commit and push
git commit -am "refactor: Remove redis deployment"
git push origin main

# 3. Watch ArgoCD prune (with prune: true)
argocd app wait h-game-redis

# 4. Verify resource deleted
kubectl get deployment redis -n h-game
# Should show: Error from server (NotFound)
```

Troubleshooting

Common Errors & Fixes

Error: Application shows "Unknown" health

Check application events:

```
argocd app get h-game-backend --show-events
```

Common causes:

- Image pull errors (check ECR permissions)
- Resource limits too low
- Health checks failing

Fix:

```
# Check pod logs
kubectl logs -n h-game deployment/backend

# Check pod status
kubectl describe pod -n h-game -l app=backend
```

Error: Application stuck in "Syncing"

Check sync status:

```
argocd app get h-game-backend
```

Common causes:

- Resource conflicts
- Permissions issues
- Health checks timing out

Fix:

```
# Check resource status
```

```
kubectl describe deployment backend -n h-game

# Check for resource conflicts
kubectl get events -n h-game --sort-by='.lastTimestamp'
```

Error: Application shows "OutOfSync" but sync doesn't happen

Check sync policy:

```
argocd app get h-game-backend
```

If automated sync is disabled, sync manually:

```
argocd app sync h-game-backend
```

Error: "CreateContainerConfigError" for database

Cause: Secret not found or not accessible

Fix:

If using Option A (plain Kubernetes Secret):

```
# Verify secret exists
kubectl get secret postgres-secret -n h-game

# If missing, check if Application synced correctly
argocd app get h-game-database

# Verify secret.yaml is in kustomization.yaml resources
cat base/database/kustomization.yaml | grep secret.yaml
```

If using Option B (AWS Secrets Manager):

```
# Verify AWS secret exists
aws secretsmanager get-secret-value --secret-id h-game/rds/password

# If using External Secrets Operator, check ExternalSecret status
```

```
kubectl get externalsecret postgres-secret -n h-game

# If using manual sync, ensure secret was created
kubectl get secret postgres-secret -n h-game

# If missing, sync manually:
SECRET_JSON=$(aws secretsmanager get-secret-value \
  --secret-id h-game/rds/password \
  --query SecretString \
  --output text)
DB_USER=$(echo $SECRET_JSON | jq -r '.username')
DB_PASSWORD=$(echo $SECRET_JSON | jq -r '.password')
DB_NAME=$(echo $SECRET_JSON | jq -r '.dbname')
kubectl create secret generic postgres-secret \
  --from-literal=postgres-user="$DB_USER" \
  --from-literal=postgres-password="$DB_PASSWORD" \
  --from-literal=postgres-db="$DB_NAME" \
  --namespace=h-game \
  --dry-run=client -o yaml | kubectl apply -f -
```

Error: "ImagePullBackOff" for frontend

Cause: Image doesn't exist in registry

Fix:

```
# For learning: Use nginx:alpine (already in manifests)
# For production: Build and push frontend image first
```

Key Takeaways

- Each component is a separate ArgoCD Application
- Health checks ensure proper startup order
- Git commits automatically deploy changes
- ArgoCD UI shows real-time deployment status
- Prune policy keeps cluster clean
- Secrets must be created before pods that use them

Bridge to G3

Single environment works. Now let's handle dev, staging, and production with different configs using Kustomize overlays...

G3: Multi-Environment Strategy

Time: 3-4 hours | **Cost:** Free

Mission Brief

Implement multi-environment deployments (dev/staging/production) with environment-specific configurations using Kustomize overlays. Learn how to manage different configurations for each environment while sharing base manifests.

Outcomes:

- Create a Kustomize base and overlays
- Deploy to dev, staging, and production environments
- Configure environment-specific settings (replicas, resources, image tags)
- Understand namespace isolation
- Manage environment promotion workflows

Prerequisites: G2 complete, all H-Game applications deployed

Before You Start: Prerequisites Checklist

G2 complete - All base manifests created and working

Kustomize understanding - Know how base + overlays work

GitOps repo accessible - Can push changes

Kubectctl configured - Can access cluster

The One Thing That Makes This Click

Kustomize overlays = environment layers on base. Just like you wear different jackets (layer) over the same shirt (base) for different weather, you apply different configs (overlays) over base manifests for different environments.

Without Overlays:

- Duplicate manifests for each environment
- Hard to maintain (change in 3 places)
- Easy to make mistakes
- No clear separation

With Overlays:

- Base manifests shared across environments
- Overlays customize for each environment
- Change base once, affects all environments
- Clear separation of concerns

Why not duplicate manifests?

- **Maintenance nightmare:** Change in 3 places (dev, staging, prod)
- **Error-prone:** Easy to forget updating one environment
- **Inconsistency:** Environments drift apart over time
- **Time-consuming:** Every change requires 3 edits

Why base + overlays?

- **DRY principle:** Define once in base, customize in overlays
- **Consistency:** All environments share the same base configuration
- **Maintainability:** Change base once, all environments benefit
- **Clarity:** Immediately see what's shared vs environment-specific

Why Kustomize (not Helm)?

- **Simplicity:** Built into kubectl (no extra tool)
- **Native:** Works with raw YAML (no templating language)
- **Flexibility:** Can patch any Kubernetes resource
- **Learning curve:** Easier for beginners than Helm templates

Why namespace isolation?

- **Resource quotas:** Different limits per environment

- **RBAC:** Different permissions per environment
- **Network policies:** Isolate environments from each other
- **Cost tracking:** Separate billing per environment

In essence, Overlays let you customize environments without duplicating code. Everything else (patches, images, namespace) is just configuration.

The Mental Model Shift:

BEFORE G3:

- "I copy manifests for dev, staging, prod"
- "I change replicas in 3 places"
- "Easy to forget to update one"

AFTER G3:

- "Base manifests shared, overlays customize"
- "Change base once, affects all"
- "Clear environment separation"

This is the difference between "duplicated configs" and "layered configs."

Now let's see how multi-environment works...

Phase 1: Understanding Multi-Environment (30 min)

Environment Differences

Common differences between environments:

Config	Dev	Staging	Production
Replicas	1	2	5
Resources	Low	Medium	High
Image	latest	specific-tag	specific-tag
Logs	debug	info	warn
Database	dev-db	staging-db	prod-db

What is Kustomize?

Kustomize is a Kubernetes configuration management tool that:

- **Patches YAML:** Modifies existing Kubernetes manifests without templating
- **Composes resources:** Combines multiple manifests into one
- **Manages variants:** Creates environment-specific configs from base
- **Built into kubectl:** No separate installation needed (`kubectl kustomize`)

How Kustomize Works:

1. Base manifests (shared configuration):

```
# base/backend/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 3 # Default: 3 replicas
  template:
    spec:
      containers:
      - name: backend
        image: backend:latest
        resources:
          requests:
            cpu: 200m
            memory: 256Mi
```

2. Kustomization file (references base):

```
# base/backend/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- deployment.yaml # Include deployment.yaml
- service.yaml    # Include service.yaml
```

3. Overlay (environment-specific changes):

```
# overlays/production/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game # Set namespace for all resources
```

```
resources:
- ../../base/backend # Reference base backend

# EASIER: Use replicas field (simpler than JSON Patch)
replicas:
- name: backend
  count: 5 # Change from 3 to 5

# EASIER: Use images field (simpler than JSON Patch)
images:
- name: backend
  newTag: v1.2.3 # Change image tag from latest to v1.2.3
```

4. Kustomize processing:

```
# Kustomize reads:
# 1. Base deployment.yaml (3 replicas, latest tag)
# 2. Overlay kustomization.yaml (patches + images)
# 3. Applies patches:
#   - Replicas: 3 → 5
#   - Image: backend:latest → backend:v1.2.3
#   - Namespace: (none) → h-game
# 4. Outputs final YAML
```

5. Result (what gets deployed):

```
# Final output after Kustomize processing
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: h-game # Added by overlay
spec:
  replicas: 5 # Changed from 3 to 5
  template:
    spec:
      containers:
```

```
- name: backend
  image: backend:v1.2.3 # Changed from latest
  resources:
    requests:
      cpu: 200m
      memory: 256Mi
```

Key Kustomize Concepts:

Resources:

- Lists YAML files or directories to include
- Can reference base directories
- Processes in order

Patches:

- JSON Patch operations (add, replace, remove)
- Target specific resources by kind and name
- Modify any field in the resource

Images:

- Change image tags without editing YAML
- Matches by image name (not full path)
- Useful for environment-specific versions

Namespace:

- Sets namespace for all resources
- Overrides namespace in base manifests
- Environment-specific isolation

Why Kustomize Overlays Work:

Base (shared):

```
base/backend/  
├── deployment.yaml    # replicas: 3 (default)  
└── kustomization.yaml
```

Overlay (environment-specific):

```
overlays/production/  
└── kustomization.yaml # Patch: replicas: 5
```

Result:

- **Base:** 3 replicas (shared across all environments)
- **Production overlay:** 5 replicas (production-specific)
- **Dev overlay:** 1 replica (dev-specific)
- **Staging overlay:** 2 replicas (staging-specific)

All environments share the same base, but each overlay customizes it.

Design Exercise: Environment Strategy

Scenario: You need to configure H-Game for dev, staging, and production.

Question: What should differ between environments?

Think about:

- Replica counts
- Resource limits
- Image tags (latest vs pinned)
- Config values (DB URLs, API keys)
- Namespaces

Your answer: _____

Answer Below ↓

Phase 2: Implementation (2.5 hours)

File: overlays/dev/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

# Set namespace for all resources in this overlay
namespace: h-game-dev

# Reference base manifests (shared across environments)
resources:
- ../../base/backend      # Backend deployment + service
- ../../base/frontend     # Frontend deployment + service
- ../../base/database     # Database StatefulSet + service
- ../../base/redis        # Redis deployment + service

# EASIER: Use replicas field (simpler than JSON Patch)
replicas:
- name: backend
  count: 1 # Dev needs fewer replicas
```

```

- name: frontend
  count: 1 # Dev needs fewer replicas

# Override image tags (dev uses latest for fast iteration)
images:
- name: backend      # Match image by name "backend"
  newTag: latest      # Change tag to "latest"
- name: frontend     # Match image by name "frontend"
  newTag: latest      # Change tag to "latest"

```

File: overlays/dev/deployment-patch.yaml (for environment variables)

```

# Strategic merge patch for environment variables
# This is simpler than JSON Patch for complex changes
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  template:
    spec:
      containers:
      - name: backend
        env:
        - name: LOG_LEVEL
          value: debug # Dev uses debug logging
        resources:
          requests:
            cpu: 100m # Dev doesn't need high resources
            memory: 128Mi

```

Update overlays/dev/kustomization.yaml to include patch:

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game-dev

```

```

resources:
- ../../base/backend
- ../../base/frontend
- ../../base/database
- ../../base/redis

replicas:
- name: backend
  count: 1
- name: frontend
  count: 1

images:
- name: backend
  newTag: latest
- name: frontend
  newTag: latest

# Strategic merge patches (simpler than JSON Patch)
patchesStrategicMerge:
- deployment-patch.yaml

```

Validate before applying:

```

# Preview what Kustomize will generate
kubectl kustomize overlays/dev > /tmp/dev-output.yaml

# Check replicas were changed
grep -A 2 "replicas:" /tmp/dev-output.yaml | grep -A 2 "backend"
# Should show: replicas: 1

# Check namespace was set
grep "namespace:" /tmp/dev-output.yaml | head -1
# Should show: namespace: h-game-dev

```


Create dev namespace:

```
kubectl create namespace h-game-dev  
# Or let ArgoCD create it with CreateNamespace=true
```

Step 2: Create Staging Overlay (30 min)**File: overlays/staging/kustomization.yaml**

```
apiVersion: kustomize.config.k8s.io/v1beta1  
kind: Kustomization  
  
# Staging namespace (separate from dev and prod)  
namespace: h-game-staging  
  
# Same base resources as dev  
resources:  
- ../../base/backend  
- ../../base/frontend  
- ../../base/database  
- ../../base/redis  
  
# Staging-specific replicas  
replicas:  
- name: backend  
  count: 2 # Staging: more than dev, less than prod  
- name: frontend  
  count: 2  
  
# Staging uses specific version tags (not latest)  
images:  
- name: backend  
  newTag: v1.2.3 # Pinned version for testing  
- name: frontend  
  newTag: v1.2.3 # Same version as backend
```

File: overlays/staging/deployment-patch.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  template:
    spec:
      containers:
      - name: backend
        env:
        - name: LOG_LEVEL
          value: info # Info-level logging (less verbose than dev
debug)
```

Update overlays/staging/kustomization.yaml:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game-staging

resources:
- ../../base/backend
- ../../base/frontend
- ../../base/database
- ../../base/redis

replicas:
- name: backend
  count: 2
- name: frontend
  count: 2

images:
- name: backend
  newTag: v1.2.3
```

```
- name: frontend
  newTag: v1.2.3

patchesStrategicMerge:
- deployment-patch.yaml
```

Validate:

```
kubectl kustomize overlays/staging > /tmp/staging-output.yaml
grep -A 2 "replicas:" /tmp/staging-output.yaml | grep -A 2 "backend"
# Should show: replicas: 2
```

Create staging namespace:

```
kubectl create namespace h-game-staging
```

Step 3: Create Production Overlay (30 min)

File: overlays/production/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

# Production uses default namespace (h-game)
namespace: h-game

# Same base resources (DRY principle)
resources:
- ../../base/backend
- ../../base/frontend
- ../../base/database
- ../../base/redis

# Production-specific replicas (high availability)
replicas:
- name: backend
  count: 5 # Production: 5 replicas for high availability
```

```

- name: frontend
  count: 5

# Production uses pinned versions (never latest)
images:
- name: backend
  newTag: v1.2.3 # Specific stable version
- name: frontend
  newTag: v1.2.3 # Same version as backend

```

File: overlays/production/deployment-patch.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  template:
    spec:
      containers:
      - name: backend
        env:
        - name: LOG_LEVEL
          value: warn # Warn-level logging (only errors, less
verbose)
      resources:
        requests:
          cpu: 500m # Production needs more resources
          memory: 512Mi
        limits:
          cpu: 1000m # CPU limit (prevents resource exhaustion)
          memory: 1Gi # Memory limit (prevents OOM kills)

```

Update overlays/production/kustomization.yaml:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game

resources:
- ../../base/backend
- ../../base/frontend
- ../../base/database
- ../../base/redis

replicas:
- name: backend
  count: 5
- name: frontend
  count: 5

images:
- name: backend
  newTag: v1.2.3
- name: frontend
  newTag: v1.2.3

patchesStrategicMerge:
- deployment-patch.yaml
```

Validate:

```
kubectl kustomize overlays/production > /tmp/prod-output.yaml
grep -A 2 "replicas:" /tmp/prod-output.yaml | grep -A 2 "backend"
# Should show: replicas: 5
```

Step 4: Create Environment Applications (30 min)

Dev Application

File: `argocd/applications/backend-dev.yaml`

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: backend-dev          # Application name (shows in ArgoCD UI)
  namespace: argocd         # ArgoCD manages Applications in argocd namespace
spec:
  project: default           # ArgoCD project (default allows all resources)

  # Source: Where to get manifests from
  source:
    repoURL: https://github.com/YOUR_USERNAME/h-game-gitops.git # GitOps repo
    targetRevision: main    # Git branch to monitor
    path: overlays/dev      # Path to dev overlay (Kustomize will process this)

  # Destination: Where to deploy
  destination:
    server: https://kubernetes.default.svc # Current cluster
    namespace: h-game-dev # Deploy to dev namespace

  # Sync policy: How ArgoCD should sync
  syncPolicy:
    automated:
      # Enable automatic sync
      prune: true # Delete resources removed from Git
      selfHeal: true # Revert manual cluster changes
    syncOptions:
      - CreateNamespace=true # Auto-create namespace if it doesn't exist
```

Staging Application

File: `argocd/applications/backend-staging.yaml`

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: backend-staging
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/YOUR_USERNAME/h-game-gitops.git
    targetRevision: main
    path: overlays/staging # Points to staging overlay
  destination:
    server: https://kubernetes.default.svc
    namespace: h-game-staging # Deploy to staging namespace
  syncPolicy:
    automated:
      prune: true # Auto-delete removed resources
      selfHeal: true # Auto-correct manual changes
    syncOptions:
      - CreateNamespace=true # Auto-create namespace if missing
```

Production Application

File: `argocd/applications/backend-production.yaml`

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: backend-production
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/YOUR_USERNAME/h-game-gitops.git
```

```

targetRevision: main
path: overlays/production # Points to production overlay
destination:
  server: https://kubernetes.default.svc
  namespace: h-game # Production uses default namespace
syncPolicy:
  automated:
    prune: false # Don't auto-delete (safety for production)
    selfHeal: true # Still auto-correct drift (maintain Git as
source of truth)
  syncOptions:
    - CreateNamespace=true

```

Why **prune: false** for Production?

- **Safety:** Prevents accidental deletion of production resources
- **Manual review:** Requires explicit approval before deleting
- **Rollback safety:** Keeps old resources until manually removed
- **Compliance:** Some orgs require manual deletion approval

Trade-off: Cluster may have orphaned resources if not manually cleaned.

Commit and Apply

```

cd ~/projects/h-game-gitops
git add overlays/ argocd/applications/
git commit -m "feat: Add dev/staging/production overlays"
git push origin main

# Apply applications
kubectl apply -f argocd/applications/backend-dev.yaml
kubectl apply -f argocd/applications/backend-staging.yaml
kubectl apply -f argocd/applications/backend-production.yaml

```

Validation:

```

# Verify applications created
kubectl get applications -n argocd | grep backend

```



```
# Should show: backend-dev, backend-staging, backend-production
```

Step 5: Verify Environments (20 min)

Check Dev (1 replica)

```
kubectl get pods -n h-game-dev  
# Should show 1 backend pod
```

Check Staging (2 replicas)

```
kubectl get pods -n h-game-staging  
# Should show 2 backend pods
```

Check Production (5 replicas)

```
kubectl get pods -n h-game  
# Should show 5 backend pods
```

View in ArgoCD UI

1. Open <https://localhost:8080> (port-forward if needed)
2. View all applications
3. See **backend-dev**, **backend-staging**, **backend-production**
4. All should show green (Healthy and Synced)

Phase 3: Validation (30 min)

Test Environment Isolation

Test 1: Update Dev Only

```
# 1. Update dev overlay
cd ~/projects/h-game-gitops
vim overlays/dev/kustomization.yaml
# Change backend replicas: 1 → 3

# 2. Commit and push
git commit -am "scale: Dev to 3 replicas"
git push origin main

# 3. Verify only dev changes
kubectl get pods -n h-game-dev      # 3 pods
kubectl get pods -n h-game-staging # Still 2 pods
kubectl get pods -n h-game         # Still 5 pods
```

Test 2: Update Base (Affects All)

```
# 1. Update base deployment
vim base/backend/deployment.yaml
# Add label: version: v2.0

# 2. Commit and push
git commit -am "feat: Add version label"
git push origin main

# 3. Verify all environments updated
kubectl get deployment backend -n h-game-dev -o yaml | grep version
kubectl get deployment backend -n h-game-staging -o yaml | grep
version
kubectl get deployment backend -n h-game -o yaml | grep version
# All should show version: v2.0
```

Test 3: Environment Promotion

```
# 1. Test in dev first
# (Make changes, test, commit)

# 2. Promote to staging
# Update staging overlay to use same image tag as dev
vim overlays/staging/kustomization.yaml
# Change image tag to match dev

# 3. Commit and push
git commit -am "promote: Move dev changes to staging"
git push origin main

# 4. Verify staging updated
argocd app wait backend-staging
kubectl get deployment backend -n h-game-staging -o
jsonpath='{.spec.template.spec.containers[0].image}'
```

Troubleshooting

Common Kustomize Errors

Error: "no matches for Id"

Cause: Target kind/name doesn't match

Fix:

```
# Check deployment name
kubectl get deployment backend -n h-game-dev

# Verify name in kustomization.yaml matches
cat overlays/dev/kustomization.yaml | grep -A 2 "replicas:"
```

Error: "invalid path"

Cause: Strategic merge patch path is wrong

Fix:

```
# Preview Kustomize output to see what's wrong
kubectl kustomize overlays/dev > /tmp/output.yaml
cat /tmp/output.yaml

# Check patch file syntax
cat overlays/dev/deployment-patch.yaml
```

Error: "image not found"

Cause: Image tag doesn't exist in registry

Fix:

```
# Verify image exists
aws ecr describe-images --repository-name h-game-backend

# Check image tag in overlay
cat overlays/production/kustomization.yaml | grep newTag
```

Error: Overlay doesn't apply patches

Check kustomization.yaml syntax:

```
kubectl kustomize overlays/dev
```

Common issues:

- Wrong target kind/name
- Incorrect patch path
- Missing resources in kustomization.yaml

Fix:

```
# Validate with kustomize build
kubectl kustomize overlays/dev > /tmp/output.yaml
# Check if patches are applied
cat /tmp/output.yaml | grep -A 5 "replicas:"
```

Error: Namespace mismatch**Check namespace in overlay:**

```
cat overlays/dev/kustomization.yaml | grep namespace
```

Common issue: Namespace in overlay doesn't match Application destination

Fix: Ensure namespace matches Application destination

Image Tagging Strategy

Dev: Uses **latest (fast iteration)**

```
# Build and push with latest tag
docker tag h-game-backend:latest $ECR_REGISTRY/h-game-backend:latest
docker push $ECR_REGISTRY/h-game-backend:latest
```

Staging/Prod: Uses Git SHA or version tag

```
# Get version from Git
VERSION=$(git rev-parse --short HEAD)
# OR use semantic version
VERSION=v1.2.3
```

```
# Tag and push
docker tag h-game-backend:latest
$ECR_REGISTRY/h-game-backend:$VERSION
docker push $ECR_REGISTRY/h-game-backend:$VERSION

# Then update overlay kustomization.yaml with newTag: $VERSION
vim overlays/staging/kustomization.yaml
# Change: newTag: v1.2.3
git commit -am "chore: Update staging to v1.2.3"
git push origin main
```

Key Takeaways

- Base manifests are shared across environments
- Overlays add environment-specific changes
- Use **replicas** and **images** fields (simpler than JSON Patch)
- Strategic merge patches for complex changes
- Each environment is isolated (separate namespaces)
- Production has stricter sync policies (**prune: false**)
- Changes to base affect all environments
- Always validate with **kubectl kustomize** before applying

Bridge to G4

Multi-environment works. Now let's add advanced patterns like sync waves and deployment hooks for ordered deployments...

G4: Advanced GitOps Patterns

Time: 2-3 hours | **Cost:** Free

Mission Brief

Implement advanced ArgoCD patterns: sync waves for ordered deployment, pre-sync hooks for migrations, rollback strategies, and progressive delivery. Learn how to orchestrate complex deployments with dependencies and hooks.

Outcomes:

- Configure sync waves for ordered deployment
- Create pre-sync and post-sync hooks
- Implement rollback strategies
- Understand progressive delivery patterns
- Manage deployment dependencies

Prerequisites: G3 complete, multi-environment setup working

- ☐ **G2 complete** - Base manifests exist and working
- ☐ **G3 complete** - Overlays exist for dev/staging/production
- ☐ **ArgoCD applications deployed** - All apps showing "Healthy" and "Synced"
- ☐ **Health endpoints working** - `/health` endpoint responds on backend
- ☐ **kubectl configured** - Can access cluster
- ☐ **ArgoCD CLI configured** - Can run `argocd` commands

Verify prerequisites:

```
# Check applications are healthy
argocd app list
# All should show: Healthy | Synced

# Check backend health endpoint
kubectl run test --image=curlimages/curl --rm -i --restart=Never -n
h-game -- \
  curl -f http://backend-service.h-game.svc.cluster.local:3001/health
# Should return: 200 OK
```

The One Thing That Makes This Click

Sync waves = orchestrated deployment. Just like an orchestra plays instruments in order (drums first, then bass, then melody), sync waves deploy resources in order (database first, then backend, then frontend).

Without Sync Waves:

- Resources are deployed in random order
- Backend might start before database
- Race conditions and failures
- No control over deployment order

With Sync Waves:

- Resources deploy in specified order
- Database starts before backend
- Frontend starts after backend
- Predictable, reliable deployments

Why not deploy everything at once?

- **Dependencies:** Backend needs the database to be ready first
- **Race conditions:** Backend might try to connect before database is up
- **Failures:** Random order causes intermittent failures
- **Unpredictability:** Can't guarantee startup sequence

Why ordered deployment?

- **Reliability:** Database always starts before backend
- **Predictability:** Same order every time (no surprises)
- **Debugging:** Easier to troubleshoot (know what starts when)
- **Production-ready:** Matches real-world deployment needs

Why use annotations (not separate Applications)?

- **Simplicity:** One Application, controlled order
- **Atomicity:** All resources in one sync operation
- **Visibility:** See the entire deployment in one view
- **Flexibility:** Easy to adjust order (just change annotation)

Why negative waves for secrets?

- **Security:** Secrets must exist before anything uses them
- **Dependencies:** All resources depend on secrets
- **Best practice:** Industry-standard pattern (secrets first)

In essence, Sync waves control when resources deploy. Everything else (hooks, rollback, progressive delivery) builds on this foundation.

The Mental Model Shift:

BEFORE G4:

- "Resources deploy randomly"
- "Hope database starts first"
- "Race conditions cause failures"

AFTER G4:

- "Sync waves control order"
- "Database always starts first"
- "Predictable deployments"

This is the difference between "hoping it works" and "ensuring it works."

Now let's see how advanced patterns work...

Phase 1: Understanding Advanced Patterns (30 min)

Sync Waves

Sync waves control deployment order:

```
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "1" # Deploy in wave 1
```

Lower numbers deploy first:

- Wave -1: Secrets (before everything)
- Wave 0: Database, Redis (stateful)
- Wave 1: Backend (depends on DB)
- Wave 2: Frontend (depends on backend)

Sync Hooks

Hooks run at specific times:

- **PreSync:** Run before sync (migrations, backups)
- **Sync:** Normal resources (deployments, services)
- **PostSync:** Run after sync (tests, notifications)
- **SyncFail:** Run on failure (rollback, alerts)

Why PreSync hooks?

- **Database migrations:** Must run before new code deploys
- **Backups:** Create a backup before risky changes
- **Validation:** Verify prerequisites before deployment
- **Safety:** Prevent deployment if the hook fails (fail fast)

Why PostSync hooks?

- **Smoke tests:** Verify deployment actually works
- **Health checks:** Ensure application is responding
- **Notifications:** Alert team on deployment success
- **Integration tests:** Run tests against deployed application

Why not just wait for Deployment to be ready?

- **Deployment ready $\neq$ Application ready:** Pod running doesn't mean app working
- **Database migrations:** Need to run before app can work
- **External dependencies:** May need to verify external services
- **Custom validation:** Business logic checks beyond Kubernetes health checks

Why delete hooks after completion?

- **Cleanliness:** Don't clutter cluster with completed jobs
- **Resource usage:** Old jobs consume resources unnecessarily
- **Visibility:** Only see active/recent hooks
- **Best practice:** HookSucceeded policy keeps cluster clean

Design Exercise: Deployment Order

Scenario: You need to deploy H-Game with the proper startup order.

Question: What's the correct sync wave order?

Think about:

- Secrets must be first
- Database must be before backend
- Redis must be before backend
- Backend must be before frontend

Your answer: _____

Answer Below 

Answer:**Sync Wave Order:**

1. **Wave -1:** Secrets (postgres-secret, redis-secret)
2. **Wave 0:** Database, Redis (stateful services)
3. **Wave 1:** Backend (depends on DB and Redis)
4. **Wave 2:** Frontend (depends on Backend)

Implementation:

- Add sync-wave annotations to resources
- Lower numbers deploy first
- ArgoCD respects wave order

Phase 2: Implementation (2 hours)

Step 1: Add Sync Waves

Time: 30 min

Database (wave 0):

File: `base/database/statefulset.yaml`

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: h-game
  annotations:
    argocd.argoproj.io/sync-wave: "0" # Deploy first
  labels:
    app: postgres
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          env:
            - name: POSTGRES_DB
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: postgres-db
```

```

- name: POSTGRES_USER
  valueFrom:
    secretKeyRef:
      name: postgres-secret
      key: postgres-user
- name: POSTGRES_PASSWORD
  valueFrom:
    secretKeyRef:
      name: postgres-secret
      key: postgres-password
ports:
- containerPort: 5432
  name: postgres
volumeMounts:
- name: postgres-data
  mountPath: /var/lib/postgresql/data
resources:
  requests:
    cpu: 200m
    memory: 256Mi
  limits:
    cpu: 500m
    memory: 512Mi
volumeClaimTemplates:
- metadata:
    name: postgres-data
  spec:
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 10Gi

```

Redis (wave 0):**File: base/redis/deployment.yaml**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: h-game
  annotations:
    argocd.argoproj.io/sync-wave: "0" # Deploy first (parallel with
database)
  labels:
    app: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7
          ports:
            - containerPort: 6379
              name: redis
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 200m
              memory: 256Mi
```


Backend (Wave 1 - Deploy After Database/Redis)

File: `base/backend/deployment.yaml`

```
# base/backend/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: h-game
  annotations:
    # Wave 1: Deploy after database (wave 0)
    # Backend depends on database, so it must wait
    argocd.argoproj.io/sync-wave: "1"
  labels:
    app: backend
spec:
  # ... rest of deployment (replicas, containers, etc.)
```

Frontend (Wave 2 - Deploy Last)

File: `base/frontend/deployment.yaml`

```
# base/frontend/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: h-game
  annotations:
    argocd.argoproj.io/sync-wave: "2" # Deploy last (after backend)
  labels:
    app: frontend
spec:
  # ... rest of deployment (replicas, containers, etc.)
```

Commit to Git:

```
cd ~/projects/h-game-gitops
git add base/
git commit -m "feat: Add sync waves for ordered deployment"
git push origin main
```

Validation:

```
# Wait for ArgoCD to detect changes (up to 3 minutes with auto-sync)
# Or trigger sync manually:
argocd app sync h-game-database
argocd app sync h-game-redis
argocd app sync h-game-backend
argocd app sync h-game-frontend

# Verify sync wave annotations
kubectl get statefulset postgres -n h-game -o yaml | grep sync-wave
# Should show: argocd.argoproj.io/sync-wave: "0"

kubectl get deployment backend -n h-game -o yaml | grep sync-wave
# Should show: argocd.argoproj.io/sync-wave: "1"
```

Step 2: Add PreSync Hook (30 min)

What is a PreSync Hook?

- Runs BEFORE ArgoCD syncs resources
- Perfect for database migrations
- If hook fails, sync stops (deployment doesn't happen)

Hook Lifecycle:

1. ArgoCD creates Job resource
2. Job runs to completion
3. If successful: ArgoCD continues sync
4. If failed: ArgoCD stops sync (deployment fails)
5. With **HookSucceeded** policy: Job deleted after success

Option A: Database Migration Hook (If You Have Migrations)

File: **base/database/migration-hook.yaml**

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration
  namespace: h-game
  annotations:
    # PreSync hook: Runs BEFORE ArgoCD syncs resources
    argocd.argoproj.io/hook: PreSync

    # Delete hook job after successful completion
    # Keeps cluster clean (no completed jobs cluttering namespace)
    argocd.argoproj.io/hook-delete-policy: HookSucceeded
spec:
  template:
    spec:
      containers:
        - name: migrate
          image: migrate/migrate:latest # Database migration tool
          command:
            - migrate
```

```

- -path=/migrations          # Path to migration files
-
-database=postgresql://$(POSTGRES_USER):$(POSTGRES_PASSWORD)@$(POSTGRES_HOST):$(POSTGRES_PORT)/$(POSTGRES_DB)?sslmode=disable
- up                          # Run all pending migrations
env:
- name: POSTGRES_HOST
  value: postgres
- name: POSTGRES_PORT
  value: "5432"
- name: POSTGRES_DB
  valueFrom:
    secretKeyRef:
      name: postgres-secret
      key: postgres-db
- name: POSTGRES_USER
  valueFrom:
    secretKeyRef:
      name: postgres-secret
      key: postgres-user
- name: POSTGRES_PASSWORD
  valueFrom:
    secretKeyRef:
      name: postgres-secret
      key: postgres-password
  restartPolicy: Never      # Job should not restart
  backoffLimit: 3           # Retry up to 3 times on
failure

```

Option B: Simple Database Validation Hook (For Students Without Migrations)

File: `base/database/validation-hook.yaml`

```

apiVersion: batch/v1
kind: Job
metadata:
  name: db-validation

```

```

namespace: h-game
annotations:
  # PreSync hook: Runs BEFORE ArgoCD syncs resources
  argocd.argoproj.io/hook: PreSync

  # Delete hook job after successful completion
  argocd.argoproj.io/hook-delete-policy: HookSucceeded
spec:
  template:
    spec:
      containers:
        - name: validate
          image: postgres:15 # Use postgres image (has pg_isready)
          command:
            - sh
            - -c
            - |
              # Wait for PostgreSQL to be ready
              # This ensures database is available before backend deploys
              echo "Waiting for PostgreSQL to be ready..."
              until pg_isready -h postgres -p 5432 -U postgres; do
                echo "PostgreSQL not ready, waiting..."
                sleep 2
              done
              echo "PostgreSQL is ready!"

              # Optional: Test database connection
              export PGPASSWORD=$(cat
/var/run/secrets/kubernetes.io/serviceaccount/namespace || echo
"postgres")
              psql -h postgres -U postgres -d hgame -c "SELECT 1;" ||
exit 1
              echo "Database connection successful!"
          env:
            - name: PGPASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret

```

```

        key: postgres-password
    restartPolicy: Never
    backoffLimit: 2

```

Update database kustomization:

File: base/database/kustomization.yaml

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game

resources:
- secret.yaml
- statefulset.yaml
- service.yaml
- validation-hook.yaml # Use this if you don't have migrations
# - migration-hook.yaml # Uncomment this if you have migrations

```

Commit to Git:

```

cd ~/projects/h-game-gitops
git add base/database/
git commit -m "feat: Add database validation PreSync hook"
git push origin main

```

Validation:

```

# Trigger sync
argocd app sync h-game-database

# Watch hook job
kubectl get jobs -n h-game -w
# Should show: db-validation (or db-migration) running, then
completed

```

```
# Check hook logs
kubect1 logs -n h-game job/db-validation
# Should show: "PostgreSQL is ready!"

# Verify hook deleted (with HookSucceeded policy)
sleep 10
kubect1 get job db-validation -n h-game
# Should show: Error from server (NotFound) - hook was deleted
```

Step 3: Add PostSync Hook (30 min)

What is a PostSync Hook?

- Runs AFTER ArgoCD syncs resources
- Perfect for smoke tests and validation
- Verifies deployment actually works (not just pods running)

Backend Smoke Test Hook

File: `base/backend/smoke-test-hook.yaml`

```
apiVersion: batch/v1
kind: Job
metadata:
  name: backend-smoke-test
  namespace: h-game
  annotations:
    # PostSync hook: Runs AFTER ArgoCD syncs resources
    argocd.argoproj.io/hook: PostSync

    # Delete hook job after successful completion
    argocd.argoproj.io/hook-delete-policy: HookSucceeded
spec:
  template:
    spec:
      containers:
        - name: test
          image: curlimages/curl:latest # Lightweight curl image
          command:
```

```

- sh
- -c
- |
  # Retry loop: Try health check up to 10 times
  echo "Starting smoke test..."
  for i in {1..10}; do
    echo "Attempt $i: Checking backend health..."
    # Use full DNS name for service
    # Format:
    <service-name>.<namespace>.svc.cluster.local:<port>
    if curl -f
    http://backend-service.h-game.svc.cluster.local:3001/health; then
      echo "Health check passed!"
      exit 0 # Success: exit with code 0
    fi
    echo "Attempt $i failed, retrying in 5 seconds..."
    sleep 5 # Wait 5 seconds before retry
  done
  echo "Health check failed after 10 attempts"
  exit 1 # Failure: all attempts failed
  restartPolicy: Never
  backoffLimit: 1 # Don't retry job (only retry curl inside
  container)

```

Note: Replace **backend-service** with your actual service name:

```

# Check your service name
kubectl get svc -n h-game | grep backend
# Use the NAME column value in the hook

```

Update backend kustomization:

File: **base/backend/kustomization.yaml**

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game

```



```
resources:  
- deployment.yaml  
- service.yaml  
- smoke-test-hook.yaml
```

Commit to Git:

```
cd ~/projects/h-game-gitops  
git add base/backend/  
git commit -m "feat: Add backend smoke test PostSync hook"  
git push origin main
```

Validation:

```
# Trigger sync  
argocd app sync h-game-backend  
  
# Watch hook job  
kubectl get jobs -n h-game -w  
# Should show: backend-smoke-test running, then completed  
  
# Check hook logs  
kubectl logs -n h-game job/backend-smoke-test  
# Should show: "✅ Health check passed!"  
  
# Verify hook deleted  
sleep 10  
kubectl get job backend-smoke-test -n h-game  
# Should show: Error from server (NotFound)
```

Step 4: Add Rollback Strategy (20 min)

Rollback Options:

1. Git revert - Revert commit in Git, ArgoCD auto-syncs
2. ArgoCD CLI - `argocd app rollback <app-name> <revision>`
3. Manual sync - Sync to previous Git commit

Configure Production Application for Rollback

File: `argocd/applications/backend-production.yaml`

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: backend-production
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/YOUR_USERNAME/h-game-gitops.git
    targetRevision: main
    path: overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: h-game
  syncPolicy:
    automated:
      prune: false # Don't auto-delete (safety for production)
      selfHeal: true # Maintain Git as source of truth (revert
manual changes)
    retry:
      limit: 3 # Retry failed syncs up to 3 times
      backoff:
        duration: 5s # Wait 5 seconds before first retry
        factor: 2 # Double wait time each retry (5s → 10s → 20s)
        maxDuration: 3m # Maximum wait time between retries
```

Why **prune: false** for Production?

- Safety: Prevents accidental deletion of production resources
- Manual review: Requires explicit approval before deleting
- Rollback safety: Keeps old resources until manually removed

Why **selfHeal: true**?

- GitOps principle: Git is source of truth
- Drift prevention: Reverts manual cluster changes
- Consistency: Cluster always matches Git

Commit to Git:

```
cd ~/projects/h-game-gitops
git add argocd/applications/
git commit -m "feat: Add rollback strategy to production app"
git push origin main
```

Test Rollback:

Method 1: Git Revert (Recommended)

```
# View Git history
cd ~/projects/h-game-gitops
git log --oneline -5

# Revert last commit
git revert HEAD
git push origin main

# ArgoCD will auto-sync to previous state
argocd app wait backend-production
```

Method 2: ArgoCD CLI Rollback

```
# View application history
argocd app history backend-production

# Rollback to specific revision
```

```
argocd app rollback backend-production <revision-number>

# Or rollback to previous revision
argocd app rollback backend-production HEAD~1
```

Method 3: Sync to Specific Commit

```
# Get commit SHA
cd ~/projects/h-game-gitops
git log --oneline -5

# Update Application to point to specific commit
kubectl patch application backend-production -n argocd --type merge
-p '{
  "spec": {
    "source": {
      "targetRevision": "<commit-sha>"
    }
  }
}'

# Sync to that commit
argocd app sync backend-production
```

Phase 3: Validation (30 min)

Test Sync Wave Order

```
# 1. Delete all applications (to test fresh deployment)
kubectl delete application -n argocd --all

# 2. Recreate applications
kubectl apply -f argocd/applications/database.yaml
kubectl apply -f argocd/applications/redis.yaml
kubectl apply -f argocd/applications/backend.yaml
kubectl apply -f argocd/applications/frontend.yaml
```

```
# 3. Watch sync order
argocd app list --watch
# Or check in ArgoCD UI
```

Expected result:

- Database syncs first (wave 0)
- Redis syncs first (wave 0, parallel with database)
- Backend syncs second (wave 1, after database/redis)
- Frontend syncs last (wave 2, after backend)

Test PreSync Hook

```
# 1. Trigger sync
argocd app sync h-game-database

# 2. Check hook execution
kubectl get jobs -n h-game
# Should show: db-validation (or db-migration) job

# 3. Check hook completed
kubectl get job db-validation -n h-game
# Should show: COMPLETIONS: 1/1

# 4. Check hook logs
kubectl logs -n h-game job/db-validation
# Should show: "PostgreSQL is ready!"

# 5. Verify hook deleted (with HookSucceeded policy)
sleep 10
kubectl get job db-validation -n h-game
# Should show: Error from server (NotFound)
```

Test PostSync Hook

```
# 1. Trigger sync
argocd app sync h-game-backend

# 2. Check hook execution
kubectl get jobs -n h-game
# Should show: backend-smoke-test job

# 3. Check hook logs
kubectl logs -n h-game job/backend-smoke-test
# Should show: "✅ Health check passed!"

# 4. Verify hook deleted
sleep 10
kubectl get job backend-smoke-test -n h-game
# Should show: Error from server (NotFound)
```

Test Rollback

```
# 1. View history
argocd app history backend-production

# 2. Make a bad change (for testing)
cd ~/projects/h-game-gitops
vim overlays/production/kustomization.yaml
# Change replicas to 0 (will break deployment)

# 3. Commit and push
git commit -am "test: Break deployment for rollback test"
git push origin main

# 4. Watch deployment fail
argocd app wait backend-production
kubectl get pods -n h-game -l app=backend
# Should show: 0 pods (deployment broken)

# 5. Rollback using Git revert
```

```
git revert HEAD
git push origin main

# 6. Watch ArgoCD restore
argocd app wait backend-production
kubectl get pods -n h-game -l app=backend
# Should show: 5 pods (deployment restored)
```

Troubleshooting

Error: Sync waves not working

Check annotations:

```
kubectl get deployment backend -n h-game -o yaml | grep sync-wave
```

Common issues:

- Annotation not applied (check Git commit)
- Wrong annotation name (must be **argocd.argoproj.io/sync-wave**)
- Annotation in wrong place (must be in **metadata.annotations**)

Fix:

```
# Verify annotation in Git
cd ~/projects/h-game-gitops
grep -r "sync-wave" base/

# Re-apply if needed
argocd app sync h-game-backend --refresh
```

Error: Hook not running

Check hook annotations:

```
kubectl get job db-validation -n h-game -o yaml | grep hook
```

Common issues:

- Wrong hook type (must be **PreSync** or **PostSync**)
- Hook not in kustomization.yaml resources
- Hook in wrong namespace

Fix:

```
# Verify hook is in kustomization.yaml
cd ~/projects/h-game-gitops
cat base/database/kustomization.yaml | grep validation-hook

# Check hook exists in cluster
kubectl get job db-validation -n h-game

# If missing, trigger sync
argocd app sync h-game-database
```

Error: Hook fails**Check hook logs:**

```
kubectl logs -n h-game job/db-validation
```

Common causes:

- Database not ready (hook runs too early)
- Wrong service name in PostSync hook
- Missing secrets

Fix:

```
# For PreSync hook: Ensure database is running
kubectl get pods -n h-game -l app=postgres

# For PostSync hook: Check service name
kubectl get svc -n h-game

# Update hook with correct service name
```


Error: Rollback fails

Check application history:

```
argocd app history backend-production
```

Common issues:

- No history available (first deployment)
- Revision not found
- Sync policy prevents rollback

Fix:

```
# Use Git revert for manual rollback
cd ~/projects/h-game-gitops
git log --oneline -5
git revert <commit-sha>
git push origin main
```

Key Takeaways

- Sync waves control deployment order (lower numbers first)
- PreSync hooks run before deployment (perfect for migrations)
- PostSync hooks run after deployment (perfect for smoke tests)
- Hooks are deleted after success (with **HookSucceeded** policy)
- Rollback is simple (Git revert or ArgoCD CLI)
- Git is always source of truth (selfHeal maintains consistency)

G5: Production GitOps Workflow (CI/CD Integration)

Time: 2-3 hours | **Cost:** Free

Mission Brief

GitHub Actions handles building and testing the code, while **ArgoCD** automatically deploys it using GitOps. Together, they create a fully automated CI/CD workflow where code changes trigger deployments automatically.

Integrate GitHub Actions (Section 7) with ArgoCD to create a complete CI/CD + GitOps workflow. CI builds and tests, ArgoCD deploys. Learn how to update the GitOps repository from the CI/CD pipeline and trigger automatic deployments.

Outcomes:

- Update GitOps repo from GitHub Actions
- Configure ArgoCD for automatic sync
- Add deployment annotations
- Understand CI/CD + GitOps separation
- Create complete automation pipeline

Prerequisites: G4 complete, Section 7 (P1-P4) complete, GitHub Actions workflow exists

Before You Start: Prerequisites Checklist

Section 7 (P1-P4) complete - GitHub Actions workflows exist

G2 complete - Base manifests in GitOps repo

G3 complete - Overlays exist

G4 complete - Sync waves and hooks working

GitOps repo separate - Different repo from application code

Personal Access Token created - For cross-repo access (see setup below)

Verify prerequisites:

```
# Check GitHub Actions workflows exist
ls -la .github/workflows/
# Should show: build.yml, deploy.yml, etc.
```

```
# Check GitOps repo is separate
cd ~/projects/h-game-gitops
git remote -v
# Should show different repo URL than application repo
```

Create Personal Access Token (Required):

1. GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic)
2. Click "Generate new token (classic)"
3. Name: **GitOps Integration**
4. Select scopes: **repo** (full control of private repositories)
5. Click "Generate token"
6. Copy the token (you won't see it again!)
7. Go to your application repo → Settings → Secrets and variables → Actions
8. Click "New repository secret"
9. Name: **GITOPS_PAT**
10. Value: Paste the token
11. Click "Add secret"

Important: Built-in **GITHUB_TOKEN** only works for the same repo. For cross-repo access (application repo → GitOps repo), you need a Personal Access Token.

The One Thing That Makes This Click

CI/CD + GitOps = complete automation. Just like a factory has both an assembly line (CI/CD builds) and quality control (ArgoCD deploys), your workflow has both build automation and deployment automation working together.

Without Integration:

- CI builds images
- Manual deployment step
- No connection between build and deploy
- Hard to track what's deployed

With Integration:

- CI builds images and updates Git
- ArgoCD detects changes and deploys
- Complete automation from code to cluster
- Full traceability (Git commit → deployment)

In essence, CI/CD handles building, and GitOps handles deploying. Together, they create complete automation. Everything else (annotations, webhooks, notifications) is just polish.

The Mental Model Shift:**BEFORE G5:**

- "CI builds, I deploy manually"
- "No connection between build and deploy"
- "Hard to track what's running"

AFTER G5:

- "CI builds and updates Git"
- "ArgoCD deploys automatically"
- "Complete automation pipeline"

This is the difference between "semi-automated" and "fully automated."

Now let's see how CI/CD + GitOps integration works...

Phase 1: Understanding CI/CD + GitOps (30 min)

CI/CD vs GitOps Separation

CI/CD (GitHub Actions) does:

- Build code
- Run tests
- Build Docker images
- Push images to ECR
- Update image tags in Git

GitOps (ArgoCD) does:

- Monitor Git for manifest changes
- Deploy to Kubernetes
- Health checks
- Rollback on failure

Key Principle: CI never touches Kubernetes. Only Git.

Image Tag Strategy

Recommended: Git SHA

- **Why Git SHA?**
 - **Immutability:** Each commit = unique tag (can't overwrite)
 - **Traceability:** Tag directly links to Git commit
 - **Simplicity:** No version management needed
 - **GitOps alignment:** Commit SHA in Git matches image tag

Example:

```
# CI builds: h-game-backend:a1b2c3d (Git SHA)
# CI updates GitOps: image: h-game-backend:a1b2c3d
# ArgoCD deploys: Uses image from Git
```

Alternative: Semantic Version

- Cleaner tags (v1.2.3)
- Requires version management

- Good for releases

Why this separation?

Security benefits:

- **Least privilege:** CI/CD doesn't need cluster access (reduces attack surface)
- **Audit trail:** All deployments go through Git (full history)
- **Access control:** Only ArgoCD needs cluster access (single point of control)
- **Compliance:** Easier to meet security requirements (no CI credentials in cluster)

Operational benefits:

- **Separation of concerns:** CI builds, GitOps deploys (clear responsibilities)
- **Rollback:** Git revert is easier than CI/CD rollback
- **Visibility:** Git history shows all deployments (who, when, what)
- **Team workflow:** Developers use familiar Git workflows

Reliability benefits:

- **Resilience:** CI/CD can fail without affecting deployments (ArgoCD keeps monitoring)
- **Consistency:** All deployments go through the same process (Git → ArgoCD)
- **Traceability:** Every deployment is linked to a Git commit (full audit trail)
- **Self-healing:** ArgoCD corrects drift even if CI/CD is down

Design Exercise: Image Tag Strategy

Scenario: You need to tag images for GitOps deployment.

Question: What tagging strategy should you use?

Think about:

- Git SHA: `h-game-backend:a1b2c3d` (immutable)
- Semantic version: `h-game-backend:v1.2.3` (clean)
- Branch name: `h-game-backend:main-a1b2c3d` (both)

Your answer: _____

Answer Below 

Answer:**Recommended: Git SHA****Why Git SHA over semantic versioning?**

- **Immutability:** Each commit = unique tag (can't overwrite)
- **Traceability:** Tag directly links to Git commit (know exactly what code)
- **Simplicity:** No version management needed (Git handles it)
- **GitOps alignment:** Commit SHA in Git matches image tag (perfect sync)

Why Git SHA over latest?

- **Reproducibility:** Can always rebuild exact same image
- **Debugging:** Know exactly which code is running
- **Rollback:** Easy to rollback to specific commit
- **Safety:** Prevents accidental deployment of wrong code

Why Git SHA over branch name?

- **Uniqueness:** Branch names can change, SHAs are permanent
- **Accuracy:** SHA is exact commit, branch might have new commits
- **Production-ready:** Industry standard for production deployments
- **Audit:** Every deployment tied to specific commit (compliance)

Alternative: Semantic Version

- Cleaner tags (v1.2.3)
- Requires version management
- Good for releases

Implementation:

- CI builds: **h-game-backend:\${{ github.sha }}**
- CI updates GitOps: **image: h-game-backend:\${{ github.sha }}**
- ArgoCD deploys: Uses image from Git

Phase 2: Implementation (2 hours)

Step 1: Update CI/CD Pipeline

Time: 40 min

Modify GitHub Actions workflow:

File: `.github/workflows/deploy.yml`

```
name: Build and Update GitOps

on:
  push:
    branches: [main]

env:
  ECR_REGISTRY: ${ secrets.ECR_REGISTRY }
  AWS_REGION: us-east-1

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ${ env.AWS_REGION }

      - name: Login to ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build and push backend
        run: |
```

```

    docker build -t ${ env.ECR_REGISTRY }}/h-game-backend:${{
github.sha }} ./backend
    docker push ${ env.ECR_REGISTRY }}/h-game-backend:${{
github.sha }}
    docker tag ${ env.ECR_REGISTRY }}/h-game-backend:${{
github.sha }} ${ env.ECR_REGISTRY }}/h-game-backend:latest
    docker push ${ env.ECR_REGISTRY }}/h-game-backend:latest

- name: Build and push frontend
  run: |
    docker build -t ${ env.ECR_REGISTRY }}/h-game-frontend:${{
github.sha }} ./frontend
    docker push ${ env.ECR_REGISTRY }}/h-game-frontend:${{
github.sha }}
    docker tag ${ env.ECR_REGISTRY }}/h-game-frontend:${{
github.sha }} ${ env.ECR_REGISTRY }}/h-game-frontend:latest
    docker push ${ env.ECR_REGISTRY }}/h-game-frontend:latest

# NEW: Update GitOps repo (CI never touches cluster, only Git)
- name: Update image tag in GitOps repo
  run: |
    # Clone GitOps repository (separate from application repo)
    git clone https://${ secrets.GITOPS_PAT
}}@github.com/YOUR_USERNAME/h-game-gitops.git
    cd h-game-gitops

    # Update image tags in production overlay using sed (no
kustomize CLI needed)
    cd overlays/production

    # Update backend image tag
    sed -i "s|newTag:.*|newTag: ${ github.sha }|g"
kustomization.yaml
    # Or if using images field:
    sed -i "s|h-game-backend:.*|h-game-backend:${ github.sha
}}|g" kustomization.yaml

    # Update frontend image tag

```

```

    sed -i "s|newTag:.*|newTag: ${github.sha}|g"
kustomization.yaml

    # Commit and push changes (ArgoCD will detect this and
    deploy)
    git config user.name "GitHub Actions"
    git config user.email "actions@github.com"
    git add .
    # [skip ci] prevents infinite loop (this commit doesn't
    trigger CI)
    git commit -m "chore: Update images to ${github.sha}
[skip ci]"
    git push origin main
    env:
        GITOPS_PAT: ${secrets.GITOPS_PAT} # Personal Access
        Token (not built-in GITHUB_TOKEN)

```

Important: Replace **YOUR_USERNAME** with your GitHub username.

Why [skip ci]?

- Prevents infinite loop: GitOps update → CI triggers → GitOps update → ...
- CI checks commit message for **[skip ci]** and skips workflow
- Only needed if CI triggers on GitOps repo changes

Update kustomization.yaml format (if needed):

File: **overlays/production/kustomization.yaml**

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game

resources:
- ../../base/backend
- ../../base/frontend

replicas:

```

```

- name: backend
  count: 5
- name: frontend
  count: 5

# Images field (easier for CI/CD to update)
images:
- name: backend
  newName: YOUR_ECR_REGISTRY/h-game-backend
  newTag: latest # Will be updated by CI/CD to Git SHA
- name: frontend
  newName: YOUR_ECR_REGISTRY/h-game-frontend
  newTag: latest # Will be updated by CI/CD to Git SHA

```

Validation:

```

# Test workflow locally (simulate CI/CD)
cd ~/projects/H-Game
# Make a small change
echo "# Test" >> backend/README.md
git add backend/README.md
git commit -m "test: Trigger CI/CD"
git push origin main

# Watch GitHub Actions
# Go to: https://github.com/YOUR_USERNAME/H-Game/actions

# After workflow completes, check GitOps repo
cd ~/projects/h-game-gitops
git pull
cat overlays/production/kustomization.yaml | grep newTag
# Should show: newTag: <git-sha>

```

Step 2: Add Deployment Annotations (30 min)

Link deployments to Git commits for traceability:

File: overlays/production/deployment-annotations.yaml

```
# Strategic merge patch for annotations (simpler than JSON Patch)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  template:
    metadata:
      annotations:
        # These will be replaced by CI/CD using sed
        gitCommit: PLACEHOLDER_GIT_COMMIT
        deployedAt: PLACEHOLDER_TIMESTAMP
        deployedBy: PLACEHOLDER_DEPLOYED_BY
```

Update kustomization:

File: overlays/production/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: h-game

resources:
- ../../base/backend
- ../../base/frontend

replicas:
- name: backend
  count: 5
- name: frontend
  count: 5

images:
- name: backend
  newName: YOUR_ECR_REGISTRY/h-game-backend
  newTag: latest
- name: frontend
```

```

newName: YOUR_ECR_REGISTRY/h-game-frontend
newTag: latest

# Strategic merge patches (simpler than JSON Patch)
patchesStrategicMerge:
- deployment-annotations.yaml

```

Update CI/CD to set annotations:

File: `.github/workflows/deploy.yml` (add to "Update image tag" step):

```

- name: Update image tag in GitOps repo
  run: |
    git clone https://${{ secrets.GITOPS_PAT
    }}@github.com/YOUR_USERNAME/h-game-gitops.git
    cd h-game-gitops/overlays/production

    # Update image tags
    sed -i "s|newTag:.*|newTag: ${{ github.sha }}|g"
    kustomization.yaml

    # Update annotations (replace placeholders)
    sed -i "s|PLACEHOLDER_GIT_COMMIT|${{ github.sha }}|g"
    deployment-annotations.yaml
    sed -i "s|PLACEHOLDER_TIMESTAMP|$(date -u
    +"%Y-%m-%dT%H:%M:%SZ")|g" deployment-annotations.yaml
    sed -i "s|PLACEHOLDER_DEPLOYED_BY|${{ github.actor }}|g"
    deployment-annotations.yaml

    # Commit and push
    git config user.name "GitHub Actions"
    git config user.email "actions@github.com"
    git add .
    git commit -m "chore: Update images to ${{ github.sha }}"
    [skip ci]
    git push origin main

```

Validation:

```
# After CI/CD runs, check annotations
kubectl get deployment backend -n h-game -o yaml | grep annotations
-A 5
# Should show: gitCommit, deployedAt, deployedBy annotations
```

Step 3: Verify Complete Workflow (20 min)**Test complete workflow:**

```
# 1. Make code change
cd ~/projects/H-Game
echo "console.log('Deployed via GitOps!');" >> backend/src/server.js

# 2. Commit and push
git commit -am "feat: Add GitOps deployment log"
git push origin main

# 3. Watch CI/CD build (GitHub Actions)
# Go to: https://github.com/YOUR_USERNAME/H-Game/actions
# - Builds image
# - Pushes to ECR
# - Updates GitOps repo

# 4. Watch ArgoCD sync (wait up to 3 minutes for auto-sync)
argocd app wait backend-production --health --timeout 300

# 5. Verify deployment
kubectl logs -n h-game deployment/backend | grep "GitOps"
# Should see: "Deployed via GitOps!"

# 6. Check deployment annotations
kubectl get deployment backend -n h-game -o
jsonpath='{.spec.template.metadata.annotations}'
# Should show: gitCommit, deployedAt, deployedBy
```

Phase 3: Validation (30 min)

Test Complete Workflow

Test 1: Code Change Triggers Deployment

```
# 1. Make code change
cd ~/projects/H-Game
echo "console.log('GitOps test');" >> backend/src/server.js

# 2. Commit and push
git commit -am "test: Add GitOps log"
git push origin main

# 3. Watch GitHub Actions
# - Build job runs
# - Images pushed to ECR
# - GitOps repo updated

# 4. Watch ArgoCD (wait up to 3 minutes)
argocd app wait backend-production

# 5. Verify deployment
kubectl get pods -n h-game -l app=backend
# Should show new pods with updated image

# 6. Check deployment annotations
kubectl get deployment backend -n h-game -o yaml | grep annotations
-A 5
# Should show gitCommit, deployedAt, deployedBy
```

Expected result:

- GitHub Actions builds and pushes images
- GitOps repo updated with new image tags
- ArgoCD detects change and syncs
- New deployment with annotations visible

Test 2: Verify Full Audit Trail

```
# 1. Git history shows all deployments
cd ~/projects/h-game-gitops
git log --oneline -10
# Each commit is a deployment

# 2. View specific deployment
git show <commit-sha>
# Shows image tag changes

# 3. Check ArgoCD history
argocd app history backend-production
# Shows sync history with Git commits
```

Expected result:

- Git history shows every deployment
- Each commit links to code change
- ArgoCD history shows sync events
- Full traceability from code to cluster

Troubleshooting

Error: GitOps repo update fails

Check GitHub token permissions:

```
# Verify token has write access
# GitHub → Settings → Developer settings → Personal access tokens
# Check token has `repo` scope
```

Common issues:

- Token doesn't have write access
- Repository not found
- Branch protection rules

Fix:

```
# Recreate token with repo scope
# Add to GitHub Secrets: GITOPS_PAT
```

Error: sed command fails

```
Check kustomization.yaml format:
cd ~/projects/h-game-gitops
cat overlays/production/kustomization.yaml
# Verify images field exists
```

Fix:

```
# Ensure kustomization.yaml has images field
# Or update sed command to match your format
```

Error: ArgoCD doesn't sync after Git update**Check auto-sync is enabled:**

```
argocd app get backend-production | grep "Sync Policy"
# Should show: automated
```

Common issues:

- Auto-sync disabled
- Sync policy set to manual
- Application not watching correct branch

Fix:

```
# Enable auto-sync
argocd app set backend-production --sync-policy automated
```

Error: Image pull fails after GitOps update

Check image exists in ECR:

```
aws ecr describe-images --repository-name h-game-backend --image-ids  
imageTag=$GIT_SHA
```

Common issues:

- Image not pushed to ECR
- Wrong image tag
- ECR permissions

Fix:

```
# Verify image in ECR  
aws ecr list-images --repository-name h-game-backend  
  
# Check IAM permissions  
aws ecr get-login-password --region us-east-1
```

Key Takeaways

- CI/CD builds images and updates Git
- ArgoCD monitors Git and deploys to cluster
- Clear separation of concerns (CI builds, GitOps deploys)
- Full audit trail in Git history
- Deployment annotations link code to cluster
- Personal Access Token required for cross-repo access
- `[skip ci]` prevents infinite loops

Section 9 Complete

What You Built:

- ArgoCD installation and GitOps fundamentals (G1)
- Complete application deployment (G2)
- Multi-environment strategy (G3)
- Advanced patterns (sync waves, hooks, rollback) (G4)
- CI/CD + GitOps integration (G5)

Complete GitOps Workflow:

```
Developer → Git commit → GitHub Actions (build/test)
      ↓
      Update GitOps repo (image tags)
      ↓
      ArgoCD detects change
      ↓
      Deploys to Kubernetes
      ↓
      Prometheus monitors (Section 8)
```

Production Benefits:

- Git as single source of truth
- Full audit trail (who deployed what, when)
- Easy rollback (Git revert)
- Self-healing (drift correction)
- Multi-environment management
- Secure (no kubectl access needed for CI)

Next: Section 10 - Capstone Project (integrate all 9 sections)